Herramienta de simulación para dar soporte a la enseñanza de arquitectura de computadoras

Maestría en Sistemas de Información

Ruiz Jose Maria

Director: Colombani Marcelo Alberto

2024

Índice

Resumen **1**

Agradecimientos **2**

1 Introducción **3**
1.1 Justificación 5
1.2 Objetivos 6
1.3 Metodología de desarrollo 7
1.4 Organización del documento 8

2 Arquitectura de computadoras **9**
2.1 Introducción a la arquitectura de computadoras 9
2.2 Arquitecturas Von Neumann y Harvard 10
2.3 Tipos de arquitecturas 12
2.4 Repertorio de instrucciones 14
2.5 Filosofías CISC y RISC 20
2.6 Arquitectura x86 23
2.7 Lenguaje ensamblador 25

3 Simulación **28**
3.1 Introducción a la simulación 28
3.2 Simulación en la educación 29
3.3 El Formalismo DEVS (Discrete Event System Specification) 30

4 Comparativa de simuladores **33**
4.1 Estudios similares 33
4.2 Simuladores bajo análisis 34
4.3 Criterios de evaluación 34
4.4 Selección de simuladores 36
4.5 Participantes en la evaluación 36
4.6 Análisis comparativo 36
4.7 Resultados 38

5 Diseño y Construcción del Simulador **41**
5.1 Requisitos de la herramienta 41
5.2 Diseño conceptual 42

5.3 Implementación del repertorio de instrucciones 44
5.4 Tercera etapa 51
5.5 Simulación visual e interactiva 52
5.6 Gestión de interrupciones y periféricos 53
5.7 Integración de métricas de rendimiento 54
5.8 Proceso de validación 54
5.9 Portabilidad y Mantenibilidad 55

Bibliografía **56**

Índice de tablas

2.1 Comparación de Arquitecturas 14

2.2 Comparativa entre CISC y RISC 22

2.3 Hitos procesadores x86 23

2.4 Línea de Tiempo de la Evolución de la Arquitectura x86 24

2.5 Comparativa de Ensambladores x86 27

4.1 Comparativa según criterios de evaluación preestablecidos 38

5.1 Bloques funcionales principales 43

5.2 Componentes del CPU x86 43

5.3 Tabla de instrucciones y códigos de operación 44

5.4 Tabla de codificación de instrucciones 45

5.6 Tabla de Instrucciones y Codificación 46

5.11 Tabla de Instrucciones y Códigos de Operación de la Arquitectura x86 51

5.8 Tabla de codificación de instrucciones ampliado 53

Índice de figuras

2.1 Arquitectura Von Neumann 11

2.2 Arquitectura Harvard 12

2.3 Características repertorio de instrucciones 15

2.4 Modos de direccionamiento 17

2.5 Formato de instrucciones 19

2.6 Formato de instrucciones del Pentium x86 25

Resumen

Existe un consenso creciente en el uso de herramientas de simulación en la enseñanza para procesos dinámicos complejos, como las operaciones intrínsecas de la computadora, que permiten representar de forma visual e interactiva la organización y arquitectura interna de la computadora, facilitando así la comprensión de su funcionamiento por parte de los alumnos y el desarrollo de los temas por parte del docente. En este contexto, los simuladores juegan una pieza clave en el campo de la Arquitectura de Computadoras, permitiendo conectar fundamentos teóricos con la experiencia práctica, simplificando abstracciones y haciendo más rica la labor docente. La arquitectura x86 es ampliamente utilizada en computadoras de escritorio y servidores. Este documento pretende realizar una comparación de los simuladores x86 que más se adecuan en el dictado de la asignatura Arquitectura de Computadoras de la carrera Licenciatura en Sistemas, establecer los criterios de evaluación y evaluar los simuladores seleccionados de acuerdo con estos criterios.

La presente investigación surge en el marco del proyecto de investigación I/D novel PID-UNER 7065: "Enseñanza/aprendizaje de asignatura Arquitectura de Computadoras con herramientas de simulación de sistemas de cómputos". El Proyecto es llevado a cabo en la Facultad de Ciencias de la Administración de la Universidad Nacional de Entre Ríos, se vincula directamente con la asignatura Arquitectura en Computadoras que se dicta en segundo año de la carrera Licenciatura en Sistemas perteneciente a la Facultad de Ciencias de la Administración de la Universidad Nacional de Entre Ríos.

Palabras clave: x86, simulador, aprendizaje, enseñanza, arquitectura de computadoras.

Agradecimientos

Agradecimientos aquí.

Capítulo 1

Introducción

En nuestra vida cotidiana, utilizamos dispositivos como computadoras de escritorio, teléfonos y relojes inteligentes, todos ellos basados en arquitecturas específicas. Comprender su funcionamiento e interacciones nos permite diseñar y desarrollar aplicaciones más eficientes.

Es esencial que los estudiantes de Arquitectura de Computadoras comprendan tanto la estructura como el funcionamiento interno de una computadora, y adquieran experiencia práctica con ellas. Para lograrlo, es fundamental disponer de un laboratorio bien equipado con el hardware adecuado y suficiente tiempo para que los estudiantes se familiaricen con las herramientas prácticas. En este contexto, se han desarrollado numerosos simuladores que facilitan la comprensión de la estructura y el funcionamiento de las computadoras, proporcionando valiosas experiencias de aprendizaje.

Esta tesis, inscrita en la Maestría en Sistemas de Información de la Facultad de Ciencias de la Administración, está directamente vinculada con el proyecto de investigación I/D novel PID-UNER 7065, titulado "Enseñanza/aprendizaje de Arquitectura de Computadoras con herramientas de simulación de sistemas de cómputo", desarrollado en la Facultad de Ciencias de la Administración de la Universidad Nacional de Entre Ríos [1].

La asignatura Arquitectura de Computadoras forma parte del plan de estudios de la carrera de Licenciatura en Sistemas, Universidad Nacional de Entre Ríos. Su objetivo es que los estudiantes comprendan la estructura y funcionamiento de las computadoras, y la ejecución lógica de un programa a nivel de instrucciones de máquina.

El primer paso para comprender las computadoras es entender que son máquinas que toman datos del exterior, los procesan y almacenan los resultados en la memoria o los envían a dispositivos de entrada y salida.

El procesamiento se realiza a través del procesador o CPU, y es en este componente donde los estudiantes encuentran mayor complejidad y dificultades para comprender su funcionamiento.

A pesar de que es posible explicar las partes del procesador, su funcionamiento, la interacción de sus componentes y enseñar lenguaje ensamblador mediante prácticas, los estudiantes suelen tener dificultades para lograr una comprensión completa del funcionamiento.

Sin embargo, la utilización de simuladores permite afianzar los conocimientos de los temas vistos en las clases teóricas, a fin de evitar que los estudiantes desvíen su atención hacia el aprendizaje del simulador propiamente dicho, es necesario que estos sean de manejo simple, intuitivo y visualmente atractivo, simplificando su aprendizaje de su uso.

La simulación es un término de uso diario en muchos contextos: medicina, militar, entretenimiento, educación, etc., debido a que permite ayudar a comprender cómo funciona un sistema, responder preguntas como "qué pasaría si", con el fin de brindar hipótesis sobre cómo o por qué ocurren ciertos fenómenos.

Para continuar, se define simulación como el proceso de imitar el funcionamiento de un sistema a medida que avanza en el tiempo. Entonces para llevar a cabo una simulación, es necesario desarrollar previamente un modelo conceptual que representa las características o comportamientos del sistema, mientras que la simulación representa la evolución del modelo a medida que avanza en el tiempo [2][3][4].

Con los avances en el mundo digital, la simulación se ha convertido en una metodología de solución de problemas indispensable para ingenieros, docentes, diseñadores y gerentes. La complejidad intrínseca de los sistemas informáticos los hace difícil comprender y costosos de desarrollar sin utilizar simulación [3].

Muchas veces en el ámbito educativo, resulta difícil transmitir fundamentos teóricos de la organización y arquitectura interna de las computadoras debido a la complejidad de los procesos involucrados. Si sólo incorporamos los medios de enseñanza tradicionales, como ser una pizarra, un libro de texto o diapositivas, los mismos tienen una capacidad limitada para representar estos fundamentos. En consecuencia, es imprescindible un alto nivel de abstracción por parte del estudiante para desarrollar un modelo mental adecuado para capturar la organización y arquitectura interna de las computadoras [5][6][7].

Es evidente la necesidad de utilizar nuevas tecnologías como recursos didácticos y medios de transferencia de conocimiento, ya que ayudan a los estudiantes a relacionar conceptos abstractos con realidades concretas. Estas tecnologías permiten situar al estudiante en un contexto que imita aspectos de la realidad, facilitando la detección de problemáticas similares a las que podrían ocurrir en situaciones reales. Este enfoque promueve un mejor entendimiento a través del trabajo exploratorio, la inferencia, el aprendizaje por descubrimiento y el desarrollo de habilidades [8][9].

Un simulador de arquitectura es una herramienta que imita el hardware de un sistema, representando sus aspectos arquitectónicos y funciones. Permiten realizar cambios,

pruebas y ejecutar programas sin riesgo de dañar componentes ni depender de equipos físicos disponibles [10].

Algunas herramientas ofrecen una representación en forma visual e interactiva de la organización y arquitectura interna de la computadora, facilitando así la comprensión de su funcionamiento. En este sentido, los simuladores juegan una pieza clave en el campo de la Arquitectura de Computadores, permitiendo conectar fundamentos teóricos con la experiencia práctica, simplificando abstracciones y facilitando la labor docente [11][12][13][14].

El repertorio de instrucciones de la arquitectura x86, ampliamente utilizado en computadoras de escritorio y servidores, comenzó con el procesador Intel 8086 en 1978 como una arquitectura de 16 bits. Evolucionó a una arquitectura de 32 bits con el procesador Intel 80386 en 1985 (i386 o x86-32) y posteriormente a 64 bits con las extensiones de AMD (AMD64) y su adopción por Intel (Intel 64) [15][16].

Un procesador x86-64 mantiene la compatibilidad con los modos x86 existentes de 16 y 32 bits, y permite ejecutar aplicaciones de 16 y 32 bits, como así también de 64 bits. Esta compatibilidad hacia atrás protege las principales inversiones en aplicaciones y sistemas operativos desarrollados para la arquitectura x86 [15][16][17].

Por ello, la enseñanza de la arquitectura x86 es de gran relevancia en la asignatura Arquitecturas de Computadoras debido a los diferentes temas que aborda.

Para brindar esta experiencia, es necesario un laboratorio equipado con el hardware adecuado y tiempo suficiente para que los estudiantes se familiaricen con las herramientas. Por este motivo, se han desarrollado muchos simuladores que facilitan la comprensión del funcionamiento y la estructura del computador, ofreciendo valiosas experiencias de aprendizaje [18].

1.1 Justificación

Los estudiantes y docentes de la asignatura de Arquitectura de Computadoras enfrentan múltiples desafíos a la hora de abordar los complejos conceptos teóricos inherentes a la arquitectura x86. Para los estudiantes, en particular, la introducción a la arquitectura de una computadora puede resultar abrumadora debido a la abstracción y el nivel de detalle técnico requerido. Por su parte, los docentes se ven limitados en la capacidad de ilustrar estos conceptos de manera gradual y progresiva debido a la falta de herramientas didácticas específicas para esta arquitectura. Ante estos desafíos, los simuladores juegan un papel crucial como herramientas de apoyo, al permitir la exploración y experimentación con los conceptos de forma visual e interactiva.

La necesidad de desarrollar un simulador específico para la arquitectura x86 se fundamenta en las limitaciones de los simuladores actuales, los cuales no permiten una

adecuada personalización al plan de estudios de la asignatura de Arquitectura de Computadoras en la Universidad Nacional de Entre Ríos. Aunque existen simuladores que apoyan la enseñanza de la arquitectura x86 en otros contextos [10][11], estos tienden a incluir una gran cantidad de contenidos preestablecidos. Si bien dichos contenidos son relevantes, presentar toda la especificación de la arquitectura x86 desde el inicio puede resultar abrumador y dificultar la comprensión de los estudiantes que recién se inician en este campo.

Esta tesis propone un enfoque distinto: desarrollar un simulador específico para la enseñanza de los contenidos de la currícula de Arquitectura de Computadoras. Este simulador ofrecerá una visión general de la estructura y funcionamiento de la computadora, abordando componentes clave como la CPU, la memoria, el módulo de E/S y los buses. Además, mostrará los micropasos necesarios para ejecutar el ciclo básico de una instrucción, ofrecerá un repertorio limitado de instrucciones que se irán habilitando conforme avanza el curso, y permitirá la generación y ejecución de programas en lenguaje ensamblador, tanto de manera paso a paso como en su conjunto. También incluirá la gestión de interrupciones básicas para la interacción con el teclado y la pantalla, y permitirá la comunicación con periféricos y la evaluación del rendimiento de los programas.

Contar con un simulador adaptado específicamente a los contenidos de esta asignatura no solo facilita el proceso de aprendizaje, al presentar los conceptos de manera progresiva y alineada con la currícula, sino que también permite una experiencia de aprendizaje contextualizada. Esto fomenta un aprendizaje significativo, en el cual los estudiantes pueden conectar teoría y práctica de manera efectiva a través de una herramienta diseñada para abordar de forma gradual y específica los conceptos fundamentales del curso.

Para garantizar que el simulador sea robusto, modular, flexible y fácil de modificar o ampliar, se explorará la utilización de técnicas formales de modelado y simulación, como las redes de Petri y DEVS (Discrete Event System Specification). Estas técnicas permiten una separación conceptual entre las capas de modelado y simulación, lo cual facilita tanto la comprensión del software como su adaptación. Además, estas metodologías permiten que las simulaciones escalen de forma transparente, posibilitando su ejecución en entornos de cómputo paralelo o distribuido sin necesidad de modificar el modelo, lo que representa una ventaja significativa en términos de escalabilidad [19][20][21].

1.2 Objetivos

El objetivo principal de esta tesis es construir una herramienta de simulación de la arquitectura x86 para apoyar la enseñanza de arquitectura de computadoras, enfocándose específicamente en los contenidos de la currícula de Arquitectura de Computadoras. Para lograr este objetivo, se plantean los siguientes objetivos específicos:

1. Estudiar y evaluar diferentes herramientas actuales de simulación destinadas a dar apoyo a la enseñanza de la arquitectura x86.
2. Construir una herramienta de apoyo para impartir los contenidos de la asignatura Arquitectura de Computadoras, para ello debe cumplir:
   - Una visión global de la estructura y funcionamiento de la computadora.
   - Generación y ejecución de programas en ensamblador.
   - Repertorio de instrucciones x86 reducido y habilitado progresivamente.
   - Simulación visual e interactiva de micropasos de instrucciones.
   - Gestión de interrupciones y comunicación con periféricos.
   - Medidas de rendimiento de ejecución de programas.

1.3 Metodología de desarrollo

La metodología de desarrollo de este simulador específico para la arquitectura x86 sigue una serie de etapas diseñadas para asegurar una transición lógica y efectiva entre el análisis, el diseño y la implementación de la herramienta, de manera que se ajuste al plan de estudios de la asignatura Arquitectura de Computadoras en la Universidad Nacional de Entre Ríos.

1. Análisis de requerimientos: En esta etapa inicial, se identifican y definen los objetivos específicos del simulador, así como los requerimientos técnicos y pedagógicos necesarios para su alineación con la currícula. Este análisis establece una base sólida y clara para todas las fases subsecuentes del proyecto, asegurando que el simulador cumpla con las necesidades educativas y técnicas del curso.

2. Revisión y recopilación de información: Se investiga y recopila información relevante sobre los simuladores actuales de arquitectura x86. Este paso incluye un análisis de las características, ventajas y limitaciones de los simuladores existentes, proporcionando una comprensión más profunda del contexto educativo y permitiendo identificar áreas de mejora en relación con el objetivo del proyecto.

3. Estudio comparativo: A partir de la información recopilada, se realiza un estudio comparativo detallado de las características de los simuladores disponibles en el mercado. Esta etapa busca evaluar cuáles de sus funcionalidades pueden adaptarse o modificarse y cuáles deberían excluirse, de acuerdo con los objetivos del simulador y las necesidades específicas del plan de estudios. Este análisis comparativo será clave para orientar las decisiones de diseño.

4. Diseño y planificación del simulador: Con base en los hallazgos previos, se define la arquitectura, las funcionalidades y los módulos específicos del simulador. El diseño se enfoca en facilitar el aprendizaje progresivo de los estudiantes, implementando un repertorio de instrucciones que se habiliten a medida que avanzan

en el curso. En esta etapa, se utilizan técnicas formales de modelado, como redes de Petri y DEVS (Discrete Event System Specification), para establecer una estructura modular, robusta y flexible que facilite tanto la comprensión como la modificación futura de la herramienta.

5. Construcción y desarrollo: En esta fase, se lleva a cabo la implementación del simulador de acuerdo con el diseño previamente definido. Cada funcionalidad se implementa y se verifica de forma secuencial, garantizando que cumpla con los requerimientos técnicos y pedagógicos establecidos. También se realizan pruebas parciales para asegurar la precisión y funcionalidad de cada módulo, lo que permite identificar y corregir errores tempranamente.

6. **Evaluación y ajuste**: Finalmente, se somete el simulador a una serie de pruebas con estudiantes o expertos en la materia para evaluar su efectividad en el aprendizaje de los conceptos de arquitectura de computadoras. Los resultados obtenidos en esta fase permiten realizar ajustes y optimizaciones necesarias, mejorando la herramienta y asegurando que cumpla con su propósito educativo de manera efectiva.

1.4 Organización del documento

El resto de este documento se estructura de la siguiente manera:

- En el Capítulo (2), se presenta una descripción detallada de la arquitectura x86, definiendo sus características y el conjunto de instrucciones que la componen. Esta base teórica es fundamental para comprender los aspectos que se simularán en el proyecto.
- El Capítulo (3) explora el papel de la simulación desde una perspectiva didáctica, enfatizando su valor como herramienta de apoyo en la enseñanza de Arquitectura de Computadoras. Aquí se revisan los beneficios de los simuladores en la enseñanza y los desafíos que ayudan a resolver en la formación de los estudiantes.
- En el Capítulo (4), se realiza un análisis comparativo de los simuladores actuales, evaluándolos según criterios previamente establecidos. Este análisis permite identificar las limitaciones y fortalezas de cada simulador y establecer el contexto para la propuesta de esta tesis.
- Finalmente, en el Capítulo (5), se describe el desarrollo de un simulador específico que se ajusta a los objetivos de enseñanza y aprendizaje de la arquitectura x86 en el contexto de la currícula. Este simulador está diseñado como una herramienta práctica y accesible para facilitar la comprensión de conceptos complejos en la asignatura.

Capítulo 2

Arquitectura de computadoras

Este capítulo presenta los conceptos fundamentales de la arquitectura de computadoras, abordando las filosofías de diseño CISC y RISC, la evolución de la arquitectura x86 y una introducción al lenguaje ensamblador, temas que sientan las bases para comprender el funcionamiento interno de los sistemas informáticos.

2.1 Introducción a la arquitectura de computadoras

La arquitectura de computadoras se centra en el diseño y la especificación de los componentes visibles para el programador en un sistema informático, como el conjunto de instrucciones, la organización de la memoria y los mecanismos de entrada/salida. Su objetivo principal es optimizar el rendimiento y la eficiencia del sistema. Esta disciplina se extiende desde la interacción entre el hardware y el software hasta la integración de sistemas completos, siendo fundamental para el avance de tecnologías modernas y sostenibles [13], [14].

Comprender la arquitectura de una computadora implica tener un conocimiento profundo de atributos claves como el repertorio de instrucciones (por ejemplo, arquitecturas x86 o ARM), la capacidad de procesamiento (32 o 64 bits), los mecanismos de entrada/salida, las técnicas de direccionamiento de memoria (directo o segmentado) y la gestión de la jerarquía de memoria [22]. Además, aspectos como el paralelismo y la eficiencia energética son esenciales para mejorar el rendimiento de los sistemas, así como para optimizar el desarrollo de software y sistemas operativos modernos [13], [23].

Es fundamental distinguir entre la arquitectura de computadoras, que abarca los componentes visibles al programador, y la organización de computadoras, que se enfoca en la implementación física de dicha arquitectura, incluye cómo están dispuestos

y coordinados los elementos del hardware, como la memoria, las señales de control y las unidades de procesamiento, para cumplir con los objetivos de rendimiento y funcionalidad [24], [25].

El estudio de la arquitectura de computadoras no solo permite comprender el funcionamiento interno de los sistemas informáticos, sino que también facilita su adaptación y optimización para satisfacer las demandas de nuevas tendencias tecnológicas, como la computación en la nube, la inteligencia artificial y el Internet de las cosas. La arquitectura bien diseñada es el cimiento que permite a estas tecnologías alcanzar su máximo potencial, impulsando de manera significativa la innovación. [13], [14]

Una arquitectura bien diseñada es el cimiento sobre el cual se desarrollan tecnologías modernas, permitiendo que alcancen su máximo potencial e impulsen significativamente la innovación.

Tener un conocimiento profundo de la arquitectura y la organización de los sistemas informáticos es vital para cualquier profesional en el campo de la informática. No solo habilita la creación de sistemas más eficientes y escalables, sino que también proporciona una base sólida para explorar áreas emergentes como la seguridad informática y los sistemas embebidos, sectores de creciente relevancia en la industria [23]. El dominio de la arquitectura de computadoras abre un abanico de oportunidades profesionales en áreas emergentes como la seguridad informática y los sistemas embebidos, sectores de creciente relevancia en la industria.

En resumen, el estudio de la arquitectura de computadoras es crucial no solo para entender cómo funcionan internamente los sistemas informáticos y resolver problemas de rendimiento, sino también para diseñar soluciones innovadoras en áreas clave como la inteligencia artificial, la ciberseguridad y los sistemas embebidos. Este conocimiento es esencial para abordar los retos tecnológicos del futuro y acceder a un mercado laboral que valora la capacidad de crear y optimizar sistemas cada vez más poderosos y eficientes [14].

2.2 Arquitecturas Von Neumann y Harvard

Para entender las arquitecturas modernas, es esencial conocer dos modelos históricos que han sentado las bases del diseño actual de sistemas informáticos: Von Neumann y Harvard. Estos paradigmas no solo forman el cimiento de muchas arquitecturas contemporáneas, sino que también ayudan a comprender sus diferencias, ventajas y limitaciones.

2.2.1 Arquitectura Von Neumann

La arquitectura Von Neumann, propuesta por John von Neumann en 1945, se caracteriza por utilizar un único espacio de memoria para almacenar tanto los datos como las instrucciones, y un único bus para transferir ambos tipos de información. Esta simplicidad ha favorecido su adopción generalizada, pero también trae consigo una limitación importante: el "cuello de botella de Von Neumann". Este fenómeno ocurre cuando el bus único se convierte en un factor restrictivo para la velocidad de procesamiento, especialmente en sistemas que requieren un acceso simultáneo a datos e instrucciones. Aunque esta arquitectura sigue siendo la base de muchas computadoras actuales, se han implementado optimizaciones y mejoras para mitigar sus limitaciones en los sistemas modernos [13], [14].

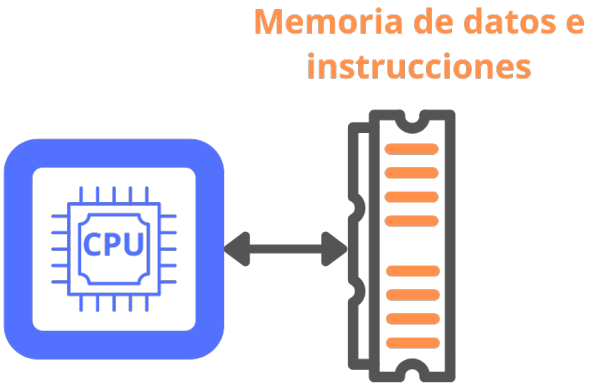

Figura 2.1: Arquitectura Von Neumann

2.2.2 Arquitectura Harvard

En contraste, la arquitectura Harvard separa físicamente las memorias de datos y de instrucciones, permitiendo que el procesador acceda a ambas de forma simultánea. Este diseño elimina el cuello de botella de Von Neumann, logrando un rendimiento más eficiente, lo que la convierte en una opción preferida para sistemas que requieren un alto desempeño, como los sistemas embebidos y los procesadores de señal digital (DSP). La arquitectura Harvard es especialmente adecuada para aplicaciones donde el acceso rápido y eficiente a los datos es crucial [24].

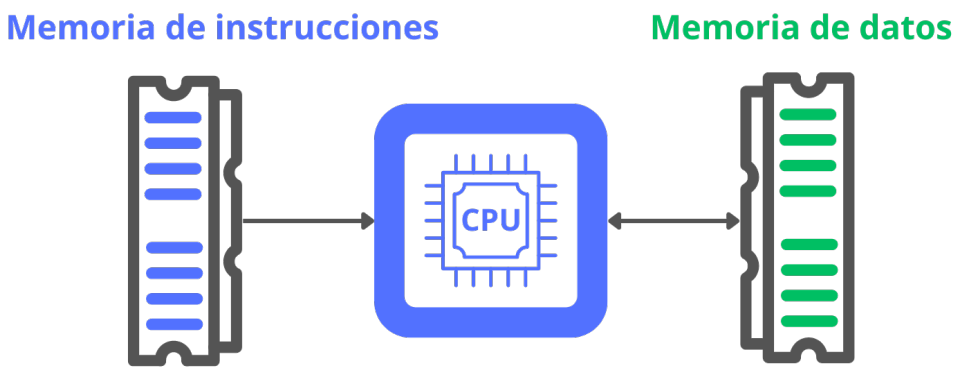

Figura 2.2: Arquitectura Harvard

2.2.3 Comparación y aplicaciones modernas

Aunque la simplicidad y flexibilidad de la arquitectura Von Neumann continúan siendo pilares fundamentales para una amplia gama de sistemas, la arquitectura Harvard ha ganado popularidad en escenarios donde la eficiencia y la velocidad son prioritarias, como en dispositivos móviles y sistemas de tiempo real. Hoy en día, arquitecturas modernas como **x86**, **ARM** y **RISC-V** implementan una combinación de ambos modelos para maximizar tanto el rendimiento como la eficiencia energética [22].

Esta integración híbrida permite a los diseñadores aprovechar las ventajas de cada enfoque, adaptando los sistemas a las necesidades específicas de las aplicaciones. La dicotomía entre ambas arquitecturas sigue siendo clave en la evolución de los sistemas modernos, donde el rendimiento y la eficiencia son factores críticos para el desarrollo de nuevas tecnologías.

2.3 Tipos de arquitecturas

El análisis de diversas arquitecturas de computadoras es esencial para comprender sus respectivas ventajas y limitaciones en diferentes contextos de aplicación. Esta evaluación comparativa permite a los diseñadores y desarrolladores de sistemas seleccionar la arquitectura más adecuada para sus necesidades, considerando factores clave como la eficiencia energética, la complejidad del hardware y las aplicaciones específicas en las que se utilizará cada una.

2.3.1 Arquitectura x86

La arquitectura x86, desarrollada inicialmente por Intel, ha dominado el mercado de computadoras de escritorio y servidores durante décadas gracias a su alto rendimiento y amplia compatibilidad con software legado. Su conjunto de instrucciones complejo (ISA) proporciona flexibilidad para una variedad de aplicaciones, aunque a costa de un mayor desafío en el diseño del hardware. Este equilibrio entre compatibilidad y rendimiento hace que x86 sea una opción preferida para entornos donde la capacidad de procesamiento es prioritaria, como en servidores y estaciones de trabajo [13].

2.3.2 Arquitectura ARM

Reconocida por su alta eficiencia energética, la arquitectura ARM es la columna vertebral de dispositivos móviles y sistemas embebidos. Basada en el paradigma de conjunto de instrucciones reducidas (RISC), ARM simplifica el diseño del hardware y optimiza el consumo energético, características que la convierten en la elección ideal para aplicaciones como smartphones y tablets. Aunque su rendimiento bruto es inferior al de x86, su balance entre eficiencia y capacidad de procesamiento es crucial para mercados donde la autonomía es esencial [23].

2.3.3 Arquitectura MIPS

MIPS, otra arquitectura basada en RISC, ha destacado históricamente en sistemas embebidos y entornos educativos debido a su simplicidad y eficacia. Aunque su relevancia ha disminuido frente a opciones como ARM, MIPS sigue siendo valiosa en nichos específicos que requieren un diseño eficiente y de bajo costo. Su conjunto de instrucciones, aunque menos versátil que el de x86, logra un equilibrio adecuado para aplicaciones de mediana complejidad [13].

2.3.4 Arquitectura RISC-V

RISC-V ha revolucionado el panorama de la arquitectura de computadoras con su enfoque abierto y libre. Su ISA flexible permite a los desarrolladores personalizar sistemas según necesidades específicas, haciéndola especialmente atractiva para investigación, educación y aplicaciones embebidas. Basada en principios RISC, RISC-V combina eficiencia energética con un diseño de hardware simplificado, y su creciente ecosistema la posiciona como una fuerte competidora frente a arquitecturas establecidas como ARM [26].

2.3.5 Compararativa de arquitecturas

Las características distintivas de cada arquitectura condicionan su idoneidad para diversas aplicaciones. Por ejemplo, mientras x86 sobresale en entornos de alto rendimiento, ARM domina en dispositivos móviles gracias a su eficiencia energética. MIPS y RISC-V, por su parte, se especializan en nichos como sistemas embebidos y proyectos personalizados. La selección adecuada de una arquitectura impacta significativamente en el éxito de un proyecto, desde el diseño hasta su implementación final [23].

Tabla 2.1: Comparación de Arquitecturas

Característica	x86	ARM	MIPS	RISC-V
Eficiencia Energética	Moderada	Alta	Moderada	Alta
Complejidad ISA	Alta	Baja	Moderada	Baja
Rendimiento	Alto	Moderado	Moderado	Variable
Compatibilidad	Alta (hacia atrás)	Moderada	Moderada	Alta
Áreas de Aplicación	Escritorio, servidores	Dispositivos móviles	Sistemas embebidos	Investigación, embebidos

2.4 Repertorio de instrucciones

El repertorio de instrucciones, conocido como ISA (Instruction Set Architecture), define el conjunto de operaciones que un procesador puede ejecutar y cómo se codifican dichas operaciones. Este conjunto incluye instrucciones aritméticas, lógicas, de control y manipulación de datos, además de los modos de direccionamiento y los formatos de las instrucciones. El diseño del ISA tiene un impacto directo en el rendimiento, la eficiencia energética y la flexibilidad del procesador, lo que lo convierte en un componente clave en la arquitectura de computadoras [13], [14], [22].

2.4.1 Características clave del ISA

Entre las características fundamentales a considerar en el diseño de un repertorio de instrucciones se encuentran las siguientes [13]:

- **Tipos de operandos**: representan los datos que las instrucciones pueden manipular, como enteros, números en punto flotante, caracteres y direcciones de memoria. Un ISA eficiente debe soportar una amplia variedad de operandos para maximizar su versatilidad.
- **Tipos de operaciones**: incluyen las operaciones que el procesador puede realizar, como aritméticas (suma, resta), lógicas (AND, OR), de control (saltos,

llamadas a subrutinas) y de manipulación de datos (almacenamiento, carga). Un repertorio bien diseñado debe equilibrar funcionalidad y simplicidad.

- **Modos de direccionamiento**: determinan cómo se especifican los operandos en las instrucciones. Modos como inmediato, directo, indirecto y relativo ofrecen diferentes niveles de flexibilidad y eficiencia.
- **Formato de las instrucciones**: define la estructura de una instrucción, incluyendo su longitud, el número de operandos y los modos de direccionamiento. Este aspecto influye tanto en la complejidad del procesador como en su rendimiento.

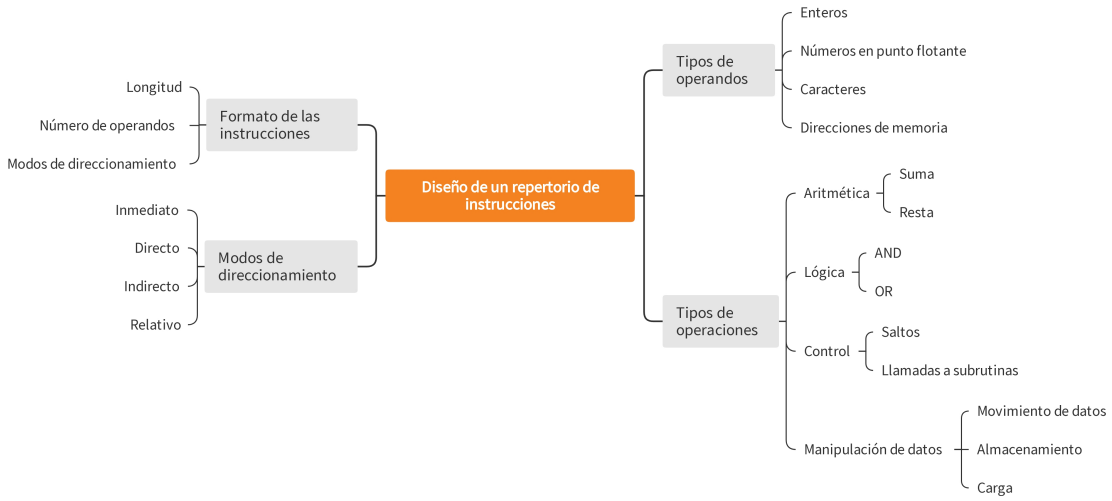

Figura 2.3: Características repertorio de instrucciones

2.4.2 Modos de direccionamiento

Los modos de direccionamiento son esenciales para determinar cómo el procesador accede a los datos necesarios para ejecutar una instrucción. Los modos más comunes incluyen [13], [14]:

a) **Inmediato**: el operando está directamente incluido en la instrucción, permitiendo acceso rápido a valores constantes. Es eficiente para operaciones simples, aunque limitado a operandos pequeños.
b) **Directo**: la instrucción contiene la dirección de memoria del operando. Es fácil de usar, pero está restringido por el rango de direcciones accesibles.
c) **Indirecto**: la instrucción apunta a una dirección que contiene la ubicación real del operando, lo que amplía el rango de direcciones a costa de un acceso adicional a memoria.

d) **Registro**: el operando se encuentra en un registro del procesador, proporcionando acceso extremadamente rápido, pero limitado por la cantidad de registros disponibles.
e) **Registro Indirecto**: similar al modo indirecto, pero la dirección se almacena en un registro, combinando rapidez y flexibilidad.
f) **Con Desplazamiento**: combina una dirección base con un valor de desplazamiento, ideal para estructuras como arrays y matrices.
g) **Pila**: el operando está en la parte superior de la pila, útil para gestionar subrutinas y el paso de parámetros.

Estos modos se ilustran en la figura 2.4 según [14]:

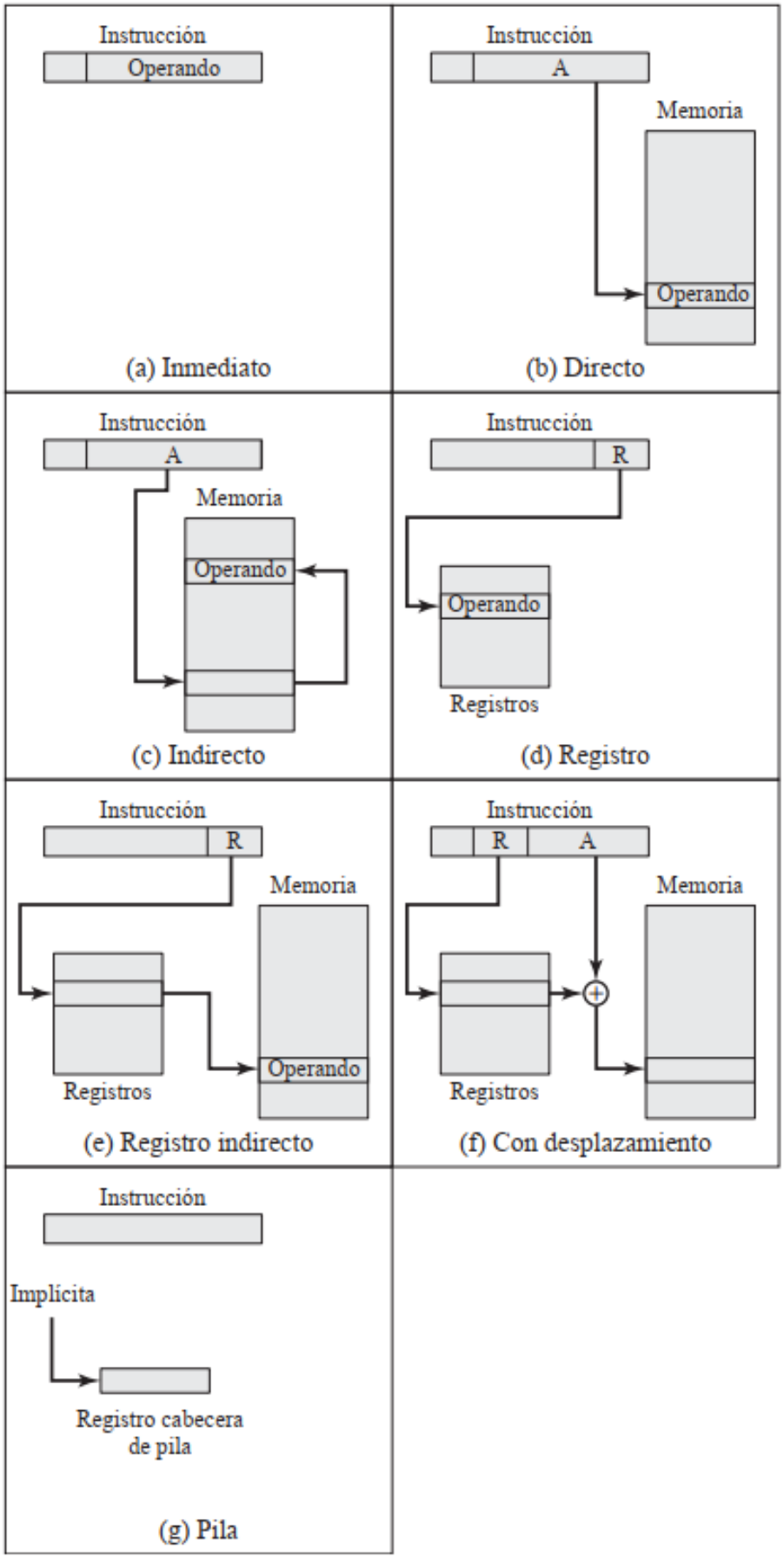

Figura 2.4: Modos de direccionamiento

- A = contenido de un campo de dirección en la instrucción
- R = contenido de un campo de dirección en la instrucción que referencia a un registro
- EA = dirección real (efectiva) de la posición que contiene el operando que se referencia

La tabla detalla el cálculo de la dirección para cada modo de direccionamiento.

\begin{table}[!h] \caption{(#tab:tab_mod_dir)Modos de direccionamiento básicos}

Modo	Algoritmo	Principal ventaja	Principal desventaja
Inmediato	Operando ← A	No referencia a memoria	Operando de magnitud limitada
Directo	EA ← A	Es sencillo	Espacio de direcciones limitado
Indirecto	EA ← (A)	Espacio de direcciones grande	Referencias múltiples a memoria
Registro	EA ← R	No referencia a memoria	Número limitado de registros
Indirecto con registro	EA ← (R)	Espacio de direcciones grande	Referencia extra a memoria
Con desplazamiento	EA ← A + (R)	Flexibilidad	Complejidad
Pila	EA ← cabecera de la pila	No referencia a memoria	Aplicabilidad limitada

\end{table}

2.4.3 Formato de las instrucciones

El formato de las instrucciones define su estructura, incluyendo longitud, cantidad de operandos y campos adicionales como el opcode. Este formato afecta la rapidez de decodificación y la eficiencia general del procesador [13], [24]:

- **Longitud de la instrucción**: puede ser fija o variable. La longitud fija simplifica el diseño del hardware, mientras que la variable optimiza el uso de memoria, aunque incrementa la complejidad de decodificación.
- **Cantidad de operandos**: las instrucciones pueden trabajar con diferentes números de operandos (de 0 a 3 o más). Más operandos ofrecen mayor versatilidad, pero aumentan la complejidad y longitud de las instrucciones.
- **Campos de instrucción**: incluyen el opcode y campos adicionales como operandos, modos de direccionamiento y flags de condición. Estos campos determinan cuántas y qué tipo de operaciones puede ejecutar el procesador en un ciclo de reloj.

La Figura ???(fig:formatoInst) ilustra un esquema típico de formato de instrucciones:

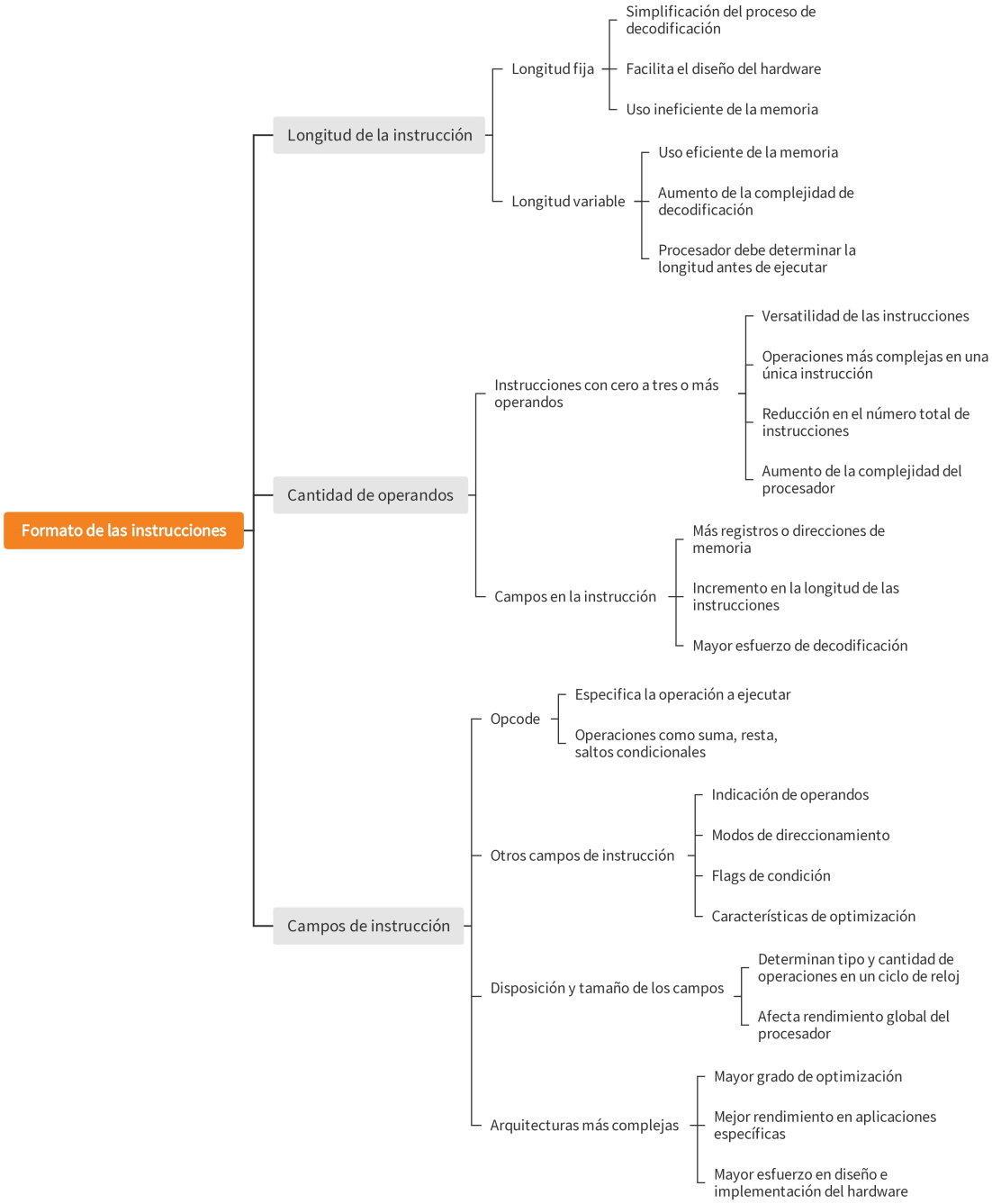

Figura 2.5: Formato de instrucciones

2.5 Filosofías CISC y RISC

El diseño del repertorio de instrucciones es un componente esencial en la arquitectura de procesadores. Dos de las filosofías más influyentes en este campo son **CISC (Complex Instruction Set Computing)** y **RISC (Reduced Instruction Set Computing)**. Mientras que **CISC** prioriza la reducción del número de instrucciones necesarias para realizar tareas complejas mediante operaciones multifuncionales, **RISC** simplifica el conjunto de instrucciones con el objetivo de maximizar la velocidad y la eficiencia energética. En esta sección se analizan ambos enfoques y sus implicaciones en el diseño de procesadores [13], [23].

2.5.1 CISC

Las arquitecturas **CISC**, como la **x86**, se caracterizan por su enfoque en reducir el número de instrucciones requeridas para completar operaciones complejas. Esto se logra mediante la inclusión de instrucciones que combinan múltiples operaciones en un solo ciclo. Como resultado, los programadores necesitan escribir menos líneas de código para alcanzar un objetivo específico.

Sin embargo, este diseño implica ciertas desventajas. La **decodificación** y **ejecución** de instrucciones CISC requiere un hardware considerablemente más complejo, y las instrucciones de longitud variable, típicas de estas arquitecturas, pueden aumentar el tiempo de decodificación. Esto genera cuellos de botella en el pipeline y limita el rendimiento.

Por ejemplo, los procesadores **x86** que ha evolucionado hacia un enfoque híbrido. Utiliza microcódigo para descomponer las instrucciones complejas en operaciones más simples, parecidas a las de un procesador RISC. Aunque esta estrategia mejora la eficiencia de ejecución en algunos casos, el diseño sigue siendo más costoso en términos de consumo energético y complejidad [23].

2.5.2 RISC

Por su parte, las arquitecturas **RISC**, como **ARM** y **MIPS**, adoptan un conjunto reducido de instrucciones de longitud fija. Esta simplificación facilita la decodificación y permite que muchas instrucciones se ejecuten en un solo ciclo de reloj. Además, esta filosofía favorece la implementación de técnicas avanzadas como el pipelining y la predicción de ramas, optimizando así el rendimiento.

A nivel de hardware, RISC prioriza la eficiencia energética, una característica crucial en dispositivos móviles y sistemas embebidos. Por ello, procesadores como los basados en ARM han dominado estos mercados. La simplicidad y el bajo CPI (ciclos por instrucción) han sido factores determinantes en su adopción [13].

2.5.3 Comparativa entre CISC y RISC

Las diferencias entre CISC y RISC son evidentes tanto a nivel de diseño como de implementación. En las arquitecturas RISC, las instrucciones tienen una longitud fija, lo que simplifica la decodificación, reduce la latencia y mejora la predictibilidad del rendimiento. Además, este formato mejora la eficiencia del uso de la memoria caché, al ocupar menos espacio y facilitar accesos más rápidos.

En cambio, las arquitecturas CISC, como x86, emplean instrucciones de longitud variable, lo que les permite ofrecer una mayor flexibilidad y un repertorio más amplio de operaciones. Sin embargo, esta flexibilidad conlleva un mayor tiempo de decodificación y una complejidad adicional en la implementación del pipeline. Esto puede causar problemas como interrupciones en el flujo debido a errores de predicción de ramas, aunque se mitiguen mediante técnicas avanzadas como la predicción dinámica de saltos y el prefetching [24].

Por ejemplo, en RISC, los modos de direccionamiento son simples y permiten un acceso más rápido a los operandos, reduciendo la latencia en el pipeline [14]. En CISC, los modos de direccionamiento más complejos proporcionan flexibilidad a costa de una mayor latencia, lo que afecta el rendimiento general del sistema.

Ejemplos de instrucciones

Para ilustrar la diferencia entre ambas filosofías, se presenta el siguiente ejemplo: cargar dos valores de memoria, sumarlos y almacenar el resultado en una dirección de memoria.

RISC:

```
; Carga el valor inmediato 10 en el registro R0
LOAD R1, [mem1]    # Cargar el valor de mem1 en el registro R1
LOAD R2, [mem2]    # Cargar el valor de mem2 en el registro R2
ADD  R3, R1, R2    # Sumar los valores en los registros R1 y R2, guardar
  en R3
STORE R3, [mem1]   # Guardar el resultado en mem1

```

CISC:

```
MOV EAX, [mem1]    # Cargar el valor de mem1 en el registro EAX
ADD EAX, [mem2]    # Sumar el valor de mem2 con EAX
MOV [mem1], EAX    # Guardar el resultado de la suma de vuelta en mem1

```

La tabla 2.2 resume las principales diferencias entre estas filosofías.

Tabla 2.2: Comparativa entre CISC y RISC

Aspecto	CISC	RISC
Objetivo principal	Minimizar el número de instrucciones para operaciones complejas	Simplificar el conjunto de instrucciones para optimizar velocidad y eficiencia energética
Tipo de instrucciones	Instrucciones complejas, longitud variable	Instrucciones simples, longitud fija
Decodificación y ejecución	Requiere hardware más complejo, posibles cuellos de botella en el pipeline	Decodificación más sencilla, facilita el uso de técnicas avanzadas como pipelining
Longitud de instrucciones	Longitud variable, puede aumentar el tiempo de decodificación	Longitud fija, simplifica la decodificación y mejora la predictibilidad del rendimiento
Eficiencia energética	Menor eficiencia energética en comparación con RISC	Mayor eficiencia energética, especialmente en dispositivos móviles
Modos de direccionamiento	Flexibilidad a costa de mayor latencia	Acceso más rápido a los operandos, menor latencia

Convergencia de filosofías

A pesar de sus diferencias, las arquitecturas modernas tienden a integrar características de ambas filosofías. Por ejemplo, los procesadores x86 adoptan técnicas propias de RISC para mejorar la eficiencia energética y el rendimiento. Esta convergencia refleja cómo los avances en diseño de procesadores buscan combinar lo mejor de cada enfoque, maximizando la flexibilidad y la eficiencia para adaptarse a las necesidades actuales y futuras del mercado.

2.6 Arquitectura x86

La arquitectura x86, ampliamente reconocida por su influencia y uso extendido en computadoras de escritorio y servidores, tuvo sus inicios en 1978 con el procesador Intel 8086, que introdujo una arquitectura de 16 bits. Desde entonces, ha experimentado una evolución continua, marcando hitos significativos en la historia de la computación. En 1985, el Intel 80386 introdujo la arquitectura de 32 bits, habilitando capacidades como la memoria virtual. Posteriormente, en 2003, AMD amplió el estándar con la arquitectura AMD64, llevando x86 a 64 bits, lo que permitió acceder a mayores espacios de memoria y mejorar el rendimiento en aplicaciones intensivas. Estas innovaciones fueron adoptadas por Intel, consolidando la arquitectura x86 como una de las más versátiles y potentes del mercado [14], [15], [16], [17].

2.6.1 Evolución de la arquitectura x86

Uno de los pilares del éxito de la arquitectura x86 ha sido su retrocompatibilidad, permitiendo la ejecución de aplicaciones de 16, 32 y 64 bits en un mismo sistema. Esta capacidad no solo ha asegurado la continuidad del ecosistema x86, sino que también ha protegido las inversiones en software y sistemas operativos, lo que resulta fundamental en el ámbito empresarial y académico.

A continuación, se presenta la tabla 2.3 que resume los hitos clave en la evolución de los procesadores x86:

Tabla 2.3: Hitos procesadores x86

Procesador	Año.de.Lanzamiento	Número.de.Bits	Nuevas.Características
Intel 8086	1978	16	Arquitectura inicial
Intel 80386	1985	32	Memoria virtual
AMD64	2003	64	Extensiones de 64 bits

La tabla 2.4 muesta comó la evolución de x86 ha estado marcada por avances tecnológicos que han impulsado la informática hacia nuevas fronteras:

Tabla 2.4: Línea de Tiempo de la Evolución de la Arquitectura x86

Año	Procesador	Innovación_Principal
1978	Intel 8086	Introducción de la arquitectura x86, 16 bits
1982	Intel 80286	Modos de operación adicionales
1985	Intel 80386	Arquitectura de 32 bits, memoria virtual
1989	Intel 80486	Unidad de punto flotante integrada, mejor caché
1993	Intel Pentium	Ejecución superescalar, predicción de saltos
1995	Intel Pentium Pro	Ejecución fuera de orden, caché L2 integrada
2003	AMD64	Extensiones a 64 bits, mayor acceso a memoria
2006	Intel Core	Optimización de rendimiento y eficiencia energética

2.6.2 Repertorio de instrucciones x86

La arquitectura x86 destaca por su complejidad y flexibilidad, reflejada en un repertorio de instrucciones extenso y de longitud variable. Esto contrasta con arquitecturas RISC, donde predominan instrucciones de longitud fija y decodificación sencilla [13]. Aunque esta flexibilidad permite adaptaciones a diversas aplicaciones, también implica mayores desafíos en términos de implementación y eficiencia operativa.

Estructura de una instrucción x86

Una instrucción típica de x86 puede incluir los siguientes componentes [14]:

- **Prefijos**: modifican la operación principal de la instrucción. Por ejemplo, el prefijo `0x66` cambia el tamaño del operando.
- **Código de operación (Opcode)**: indica la operación a realizar. Por ejemplo, `0x89` corresponde `MOV`.
- **Modificadores de dirección (ModR/M y SIB)**: definen registros y direccionamiento. El byte **SIB** (Scale, Index, Base) es especialmente útil para operaciones complejas, como el acceso a matrices.
- **Desplazamiento e inmediato**: Agregan flexibilidad en el manejo de datos, aunque aumentan la complejidad.

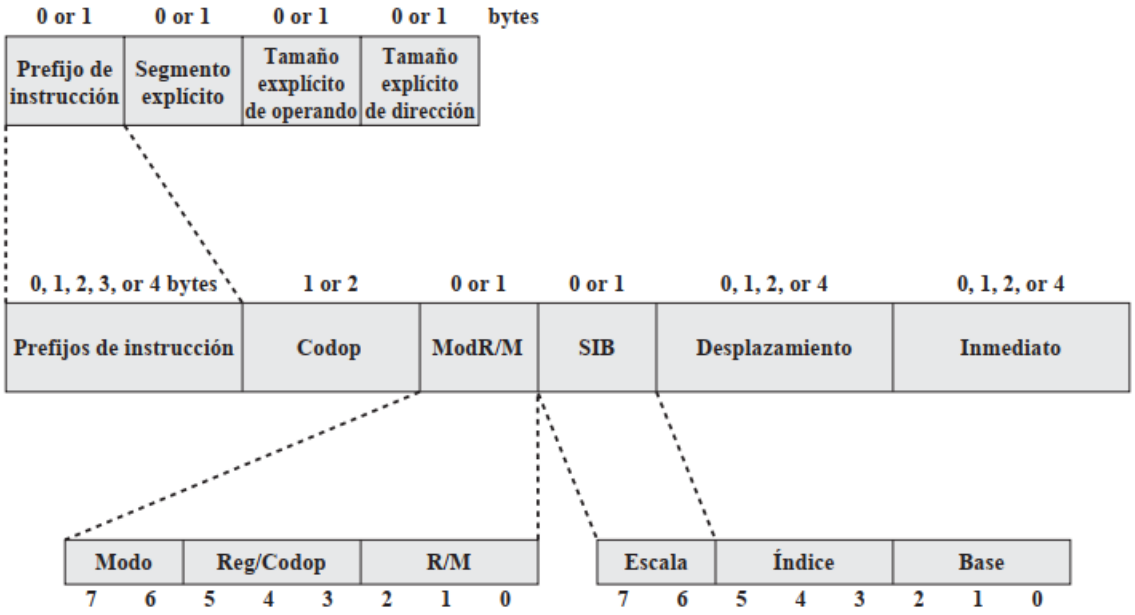

Figura 2.6: Formato de instrucciones del Pentium x86

Un ejemplo típico de instrucción es:

```
; Carga el valor inmediato 10 en el registro R0
MOV AX, [BX+SI+16]

```

Esta instrucción utiliza varios componentes, que el procesador debe decodificar antes de ejecutarla. Aunque esta flexibilidad es una ventaja en términos de funcionalidad, requiere técnicas avanzadas, como predicción de saltos y paralelización, para mantener la eficiencia en procesadores modernos [23].

2.7 Lenguaje ensamblador

Un procesador puede interpretar y ejecutar instrucciones exclusivamente en **lenguaje máquina**, que consiste en secuencias de números binarios almacenadas en memoria. Estas instrucciones son leídas y ejecutadas directamente por el procesador. Si un programador intentara escribir código en lenguaje máquina, debería especificar manualmente cada secuencia de ceros y unos para realizar operaciones, respetando estrictamente las estructuras de memoria y los modos de direccionamiento del procesador. Este proceso no solo es tedioso, sino también altamente propenso a errores, especialmente al realizar modificaciones, ya que implicaría descifrar y reescribir las secuencias binarias [27].

Para simplificar este desafío, se desarrolló el **lenguaje ensamblador**, un lenguaje de programación de bajo nivel que permite a los programadores utilizar instrucciones más legibles mediante mnemónicos simbólicos. A diferencia del lenguaje máquina, que opera con secuencias binarias, el ensamblador emplea símbolos que representan directamente las instrucciones ejecutadas por el procesador. Cada arquitectura de procesador tiene su propio lenguaje ensamblador, generalmente diseñado por el fabricante del hardware y optimizado para la arquitectura específica [14].

2.7.1 Ensamblador

El ensamblador es un programa que traduce las instrucciones simbólicas escritas en lenguaje ensamblador a lenguaje máquina, es decir, las convierte en las secuencias binarias que el procesador puede interpretar y ejecutar. Este proceso de traducción es prácticamente directo, ya que existe una correspondencia uno a uno entre las instrucciones en ensamblador y las instrucciones en lenguaje máquina [14]. En contraste, los lenguajes de programación de alto nivel, como C o Python, suelen generar múltiples instrucciones máquina por cada línea de código fuente, lo que los distancia más de la arquitectura subyacente.

Ensambladores x86

En el caso de la arquitectura x86, los programadores pueden elegir entre diversos ensambladores, como TASM (Turbo Assembler) [28], MASM (Microsoft Macro Assembler) [29] y NASM (Netwide Assembler) [30]. Aunque cada ensamblador tiene características y sintaxis particulares, todos comparten el objetivo fundamental de convertir las instrucciones ensamblador en código binario ejecutable por los procesadores x86 [31].

A continuación, se presenta una comparativa de las principales características de estos ensambladores:

Tabla 2.5: Comparativa de Ensambladores x86

Característica	**TASM**	**MASM**	**NASM**
Desarrollador	Borland	Microsoft	Simon Tatham et al.
Año de lanzamiento	1985	1981	1996
Sistema operativo	MS-DOS, Windows	MS-DOS, Windows	Multiplataforma (Windows, Linux, macOS)
Sintaxis	Sintaxis similar a Intel con extensiones	Sintaxis de Intel con soporte avanzado	Sintaxis de Intel, modular y extensible
Soporte de macros	Macros y directivas avanzadas	Macros y directivas extensivas	Macros avanzadas y preprocesamiento
Compatibilidad	Compatibilidad con x86 antiguo	Compatibilidad con x86 antiguo	Compatibilidad con x86, x86-64 y otros
Capacidades adicionales	Integración con herramientas Borland	Integración con Visual Studio	Soporte para múltiples formatos (binario, ELF, etc.)
Licencia	Comercial	Comercial	Código abierto
Uso actual	Menos común, usado en entornos heredados	Ampliamente usado en desarrollo Windows	Popular en sistemas y software libre

Capítulo 3

Simulación

En este capítulo se analiza el papel de la simulación desde una perspectiva didáctica, destacando su relevancia como herramienta de apoyo en la enseñanza de Arquitectura de Computadoras. Se abordan los beneficios que ofrecen los simuladores en el proceso educativo y los desafíos que ayudan a superar en la formación de los estudiantes.

3.1 Introducción a la simulación

La simulación es una herramienta esencial en una amplia variedad de campos, como la medicina, el ámbito militar, el entretenimiento y la educación. Su principal valor radica en su capacidad para modelar sistemas complejos, generar hipótesis, realizar análisis predictivos y explorar escenarios de manera segura y eficiente.

Banks define la simulación como el proceso de replicar el comportamiento de un sistema a lo largo del tiempo mediante un modelo conceptual que representa sus características y dinámicas principales [2]. Estos modelos evolucionan simulando las interacciones entre sus componentes, lo que permite estudiar su respuesta ante diferentes variables y escenarios [4].

La posibilidad de analizar sistemas complejos sin intervenir directamente en ellos convierte a la simulación en una herramienta indispensable en el contexto actual, marcado por el avance de la tecnología y la creciente complejidad de los sistemas. Además, la simulación permite optimizar diseños, prever comportamientos y reducir los costos de desarrollo antes de implementar soluciones reales [3], [20].

3.1.1 Aplicaciones de la simulación en la industria

En sectores como la industria automotriz, la simulación es fundamental para el diseño y prueba de sistemas de seguridad, como airbags y frenos. Gracias a modelos virtuales, se realizan pruebas de colisión y análisis de rendimiento sin necesidad de recurrir a costosas pruebas físicas. Asimismo, la simulación permite optimizar diseños de motores, analizar el flujo aerodinámico y prever el comportamiento de materiales en condiciones extremas, contribuyendo a mejorar tanto la eficiencia como la seguridad de los vehículos [32].

En la aviación, los simuladores de vuelo son esenciales para entrenar pilotos, replicando condiciones reales de vuelo sin riesgos. Durante el diseño de aeronaves, estas herramientas permiten evaluar la aerodinámica y el rendimiento en diversos entornos, reduciendo significativamente el tiempo y los costos de desarrollo mientras incrementan la seguridad [33].

Estos ejemplos destacan cómo la simulación no solo optimiza procesos industriales, sino que también asegura una mayor eficacia y seguridad en el diseño de productos complejos.

3.2 Simulación en la educación

En el ámbito educativo, la simulación ha emergido como una herramienta poderosa para facilitar la enseñanza de conceptos complejos y fomentar el aprendizaje activo. A través de simuladores, los estudiantes pueden interactuar con sistemas virtuales y experimentar escenarios realistas, lo que mejora la comprensión de ideas abstractas y favorece la aplicación práctica de conocimientos teóricos [5].

A diferencia de los métodos tradicionales, como el uso de libros de texto o diapositivas, las herramientas de simulación integran tecnologías que vinculan conceptos teóricos con situaciones reales. Esto promueve una pedagogía interactiva, basada en la resolución de problemas y el aprendizaje por descubrimiento, estimulando la exploración y el razonamiento inferencial [6].

En definitiva, la simulación enriquece la experiencia de aprendizaje al proporcionar una plataforma dinámica y participativa que facilita tanto la experimentación como la asimilación profunda de los contenidos.

3.2.1 El rol de la simulación en la enseñanza de Arquitectura de Computadoras

En la carrera de Licenciatura en Sistemas, la asignatura Arquitectura de Computadoras persigue varios objetivos esenciales: - Comprender la estructura y

funcionamiento de las computadoras. - Conocer las diferentes arquitecturas de sistemas microprocesadores. - Evaluar medidas de rendimiento y comparar arquitecturas. - Analizar el impacto de la tecnología de las computadoras en contextos sociales y económicos.

Enseñar los fundamentos teóricos de la organización y arquitectura interna de las computadoras puede ser un reto debido a la complejidad de los procesos involucrados. Los estudiantes necesitan desarrollar altos niveles de abstracción para construir modelos mentales que les permitan entender conceptos como la ejecución de instrucciones, la gestión de memoria o la interacción entre componentes del sistema.

Los simuladores desempeñan un papel crucial en este contexto al ofrecer representaciones visuales e interactivas de procesos complejos. Estas herramientas permiten a los alumnos experimentar con configuraciones y parámetros, observar su impacto en el rendimiento del sistema y explorar escenarios hipotéticos sin necesidad de hardware físico.

Además, la simulación actúa como un puente entre la teoría y la práctica, facilitando que los docentes refuercen conceptos abstractos con experiencias concretas. En conjunto, estas ventajas hacen de la simulación una metodología pedagógica invaluable, promoviendo la experimentación y el aprendizaje activo en la enseñanza de Arquitectura de Computadoras [7], [8], [18].

3.3 El Formalismo DEVS (Discrete Event System Specification)

El formalismo DEVS es una metodología modular y jerárquica que permite modelar y analizar sistemas representables como sistemas de eventos discretos, continuos o híbridos. Desarrollado por Bernard P. Zeigler en la década de 1970, este enfoque amplía el concepto de las máquinas de Moore al incorporar una estructura capaz de modelar sistemas complejos mediante eventos cronometrados y transiciones de estados [20].

3.3.1 Estructura del formalismo DEVS

DEVS describe el comportamiento de un sistema real utilizando eventos de entrada y salida, así como transiciones entre estados definidos. Un sistema en este formalismo se compone de dos tipos principales de modelos:

- **Modelos atómicos**: representan las unidades fundamentales de comportamiento.

- **Modelos acoplados**: integran modelos atómicos y/o otros modelos acoplados, permitiendo la construcción jerárquica de sistemas más complejos.

Esta organización modular facilita el análisis y la gestión de sistemas, permitiendo probar subsistemas de manera aislada antes de integrarlos en un modelo completo.

3.3.2 Aplicaciones del formalismo DEVS

El formalismo DEVS encuentra aplicación en diversos ámbitos:

- **Redes de comunicación**: permite simular el enrutamiento de paquetes de datos, gestionar la congestión y optimizar protocolos de transmisión.
- **Manufactura**: modela el flujo de materiales, el control de calidad y los procesos en líneas de producción, identificando cuellos de botella y áreas para mejora.
- **Sistemas de transporte**: simula el comportamiento de flujos de tráfico o redes de transporte público para mejorar la eficiencia y reducir costos operativos.

Estas aplicaciones destacan su versatilidad para optimizar sistemas complejos en escenarios del mundo real.

3.3.3 DEVS en la enseñanza de la Arquitectura de Computadoras

La implementación de entornos de simulación basados en DEVS en la enseñanza de arquitectura de computadoras aporta múltiples ventajas que enriquecen el proceso de aprendizaje:

- **Representación visual**: ofrece diagramas y representaciones dinámicas que ayudan a los estudiantes a visualizar y comprender procesos internos, como la ejecución de instrucciones y la gestión de recursos.
- **Interactividad**: permite modificar configuraciones y parámetros, fomentando la experimentación y mostrando el impacto directo de estas variables en el rendimiento del sistema.
- **Exploración de escenarios**: posibilita simular escenarios hipotéticos y evaluar el comportamiento de sistemas complejos sin la necesidad de hardware físico.

Estas características transforman la experiencia de aprendizaje, al vincular teoría con práctica y promover un enfoque activo en la comprensión de conceptos clave de

arquitectura de computadoras. Al adoptar DEVS como parte del entorno educativo, se potencia la capacidad de los estudiantes para abordar problemas complejos y explorar soluciones innovadoras [34].

En conclusión, el formalismo DEVS no solo es una herramienta valiosa para el modelado y análisis de sistemas, sino que también representa un recurso poderoso para facilitar la enseñanza de conceptos complejos, como los que se encuentran en la arquitectura de computadoras.

Capítulo 4

Comparativa de simuladores

En este capítulo se presentan el análisis y la comparación de diversos simuladores x86 con el propósito de evaluar su posible integración en la asignatura Arquitectura de Computadoras de la Licenciatura en Sistemas.

La selección y evaluación de estos simuladores se fundamenta en criterios específicos diseñados para medir su efectividad en un entorno educativo. El objetivo principal es identificar las herramientas que mejor respalden el proceso de enseñanza y aprendizaje. Los criterios considerados incluyen aspectos como usabilidad, funcionalidad del editor de código, calidad de la documentación, características de la ejecución de simulación, nivel de especificación de la arquitectura x86, propiedades del producto y alineación con los contenidos curriculares.

Los resultados de esta investigación fueron publicados en el XVII Congreso de Tecnología en Educación y Educación en Tecnología (2022), en el trabajo titulado Herramientas de software para dar soporte en la enseñanza y aprendizaje de la arquitectura x86 [35].

4.1 Estudios similares

Existen antecedentes de estudios comparativos que evalúan simuladores aplicados a la enseñanza en cursos de arquitectura de computadoras: - "A survey and evaluation of simulators suitable for teaching courses in computer architecture and organization", 2009 [11]: este estudio analiza simuladores considerando dos categorías principales. La primera, relacionada con las características de simulación, incluye criterios como granularidad, usabilidad, disponibilidad, presentación visual y flujo de simulación. La segunda categoría evalúa la cobertura de los contenidos establecidos en los planes de estudio. - "Survey and evaluation of simulators suitable for teaching for computer architecture and organization Supporting undergraduate

students at Sir Syed University of Engineering & Technology", 2012 [12]: este trabajo evalúa aspectos como la usabilidad, disponibilidad, fundamentos de arquitectura informática, jerarquía de sistemas de memoria, comunicación e interfaz, y diseño de sistemas de procesadores.

En contraste, este trabajo propone un enfoque diferenciado al centrar la evaluación en simuladores x86 bajo criterios diseñados específicamente para abordar tanto las características de simulación como los contenidos particulares de la asignatura Arquitectura de Computadoras de la Licenciatura en Sistemas de la Universidad Nacional de Entre Ríos.

4.2 Simuladores bajo análisis

Un simulador de arquitectura es una herramienta de software que emula el hardware de un sistema de cómputo, permitiendo representar aspectos arquitectónicos y funcionales del mismo. Estos simuladores ofrecen un entorno controlado para realizar pruebas, modificaciones y ejecución de programas sin riesgo de dañar componentes físicos o enfrentar limitaciones de hardware [10].

Algunos simuladores destacan por proporcionar una representación visual e interactiva de la organización y arquitectura interna de una computadora, facilitando la comprensión de su funcionamiento. Ejemplos de estos simuladores incluyen Assembly Debugger (x86), Simple 8-bit Assembler Simulator, Microprocessor Simulator, Simulador de ensamblador de 16 bits y Emu8086. Estas herramientas juegan un papel fundamental en el aprendizaje de la arquitectura de computadoras, al conectar conceptos teóricos con experiencias prácticas y simplificar abstracciones complejas, además de servir como soporte en la labor docente [11][12][13][14][36].

4.3 Criterios de evaluación

Los criterios de evaluación se definieron cuidadosamente para garantizar un análisis integral de los simuladores. A continuación, se presentan estos criterios junto con sus respectivos indicadores y escalas:

- **Usabilidad**: evalúa la facilidad de uso del simulador.
  - **Indicadores**:
    - Facilidad de aprendizaje (tiempo necesario para familiarizarse con la herramienta).
    - Interfaz de usuario (claridad y organización).

    * Documentación y ayuda (accesibilidad y calidad de tutoriales y guías).
  - **Escala**: Difícil - Media - Fácil.

- **Editor**: analiza las funcionalidades para escribir y depurar código ensamblador.
  - **Indicadores**:
    * Capacidad de edición (resaltado de sintaxis, puntos de interrupción, etc.).
    * Manejo de errores de sintaxis.
    * Opciones de almacenamiento (guardar y cargar programas).
  - **Escala**: Baja - Media - Alta.

- **Documentación**: valora la disponibilidad y calidad de los recursos de aprendizaje proporcionados.
  - **Indicadores**:
    * Manual de usuario.
    * Tutoriales de aprendizaje.
    * Exhaustividad en la descripción del repertorio de instrucciones.
  - **Escala**: Mínima - Media - Completa.

- **Ejecución de simulación**: mide la facilidad para controlar y observar la ejecución de programas.
  - **Indicadores**:
    * Control de simulación (pausa, reanudación, retroceso).
    * Visualización del flujo de ejecución.
    * Configurabilidad (ajuste de parámetros como la velocidad del reloj).
  - **Escala**: Baja - Media - Alta.

- **Nivel de especificación de la Organización y Arquitectura del sistema simulado**: determina la precisión en la representación de la arquitectura x86.
  - **Indicadores**:
    * Fidelidad en la representación de la arquitectura.
    * Completitud del conjunto de instrucciones implementadas.
    * Inclusión y funcionalidad de memoria y módulos de E/S.
  - **Escala**: Mínima - Media - Completa.

- **Características del producto software**: evalúa las propiedades generales del simulador.
  - **Indicadores**:
    * Tipo de licencia (open source o privativa).
    * Frecuencia de actualizaciones.
    * Plataforma (aplicación web o de escritorio)
  - **Escala**: Mala - Buena - Muy buena.

- **Cobertura de los contenidos preestablecidos en la currícula**: mide el grado en que el simulador abarca los contenidos de la asignatura.
  - **Indicadores**:
    - Alineación con los tópicos del currículum.
    - Profundidad en el tratamiento de los temas.
  - **Escala**: Baja - Media - Alta.

4.4 Selección de simuladores

A partir de una exhaustiva exploración en internet sobre herramientas de simulación de la arquitectura x86 utilizadas para la enseñanza, se identificaron los siguientes simuladores: Assembly debugger (x86), Simple 8-bit Assembler Simulator, Microprocessor Simulator, Simulador de ensamblador de 16 bits, Emu8086, VonSim, Orga1 y Qsim.

La selección se basó en una evaluación preliminar que consideró el tiempo necesario para su análisis y el grado de cumplimiento de los criterios definidos, priorizando aquellos simuladores que ofrecieran un balance adecuado entre funcionalidad, usabilidad, documentación y alineación con los contenidos curriculares de la asignatura Arquitectura de Computadoras. De esta preselección, se eligieron tres herramientas que, a priori, cumplían con la mayor cantidad de criterios evaluativos: **Emu8086**, **VonSim** y **Simple 8-bit Assembler Simulator**.

4.5 Participantes en la evaluación

El proceso de evaluación fue realizado por un equipo conformado por tres docentes de la asignatura Arquitectura de Computadoras —Marcelo A. Colombani, José M. Ruiz y Amalia G. Delduca—, quienes aportaron su experiencia en el uso de simuladores y su aplicabilidad pedagógica. Además, se contó con la colaboración de un asesor externo, Marcelo A. Falappa, quien ofreció una visión independiente y validó tanto el proceso de evaluación como los resultados obtenidos.

4.6 Análisis comparativo

A continuación, se presenta un análisis detallado de los simuladores seleccionados, basado en los criterios previamente establecidos:

4.6.1 Simple 8-bit Assembler Simulator

- **Usabilidad**: Nivel medio. Todos los componentes se muestran en una sola pantalla, lo que puede resultar abrumador para usuarios principiantes.
- **Editor**: Nivel bajo. Incluye notificaciones de errores de sintaxis al ensamblar, pero carece de resaltado de sintaxis, puntos de interrupción (breakpoints) y opciones para guardar o cargar programas.
- **Documentación**: Nivel mínimo. Consta solo de un manual de instrucciones implementadas.
- **Ejecución de simulación**: Nivel medio. Permite ajustar la velocidad del reloj de la CPU y proporciona controles básicos de simulación.
- **Nivel de especificación**: Nivel mínimo. Simplifica la arquitectura x86 a un CPU de 8 bits con 256 bytes de memoria y sin soporte para operaciones de entrada/salida (IN/OUT).
- **Desarrollo del producto**: Nivel bueno. Licencia MIT, última actualización en 2015, desarrollado como una plataforma web.
- **Cobertura de contenidos**: Nivel bajo. No incluye memoria independiente para módulos de entrada y salida, rutinas de interrupciones ni representación visual del ciclo de instrucción.

4.6.2 VonSim

- **Usabilidad**: Nivel medio. Utiliza solapas para presentar los componentes, lo que puede ser confuso para usuarios iniciales.
- **Editor**: Nivel medio. Proporciona notificaciones de errores de sintaxis, resaltado de código y puntos de interrupción mediante software.
- **Documentación**: Nivel medio. Incluye un manual de uso y un tutorial interactivo.
- **Ejecución de simulación**: Nivel medio. Permite ajustar la velocidad del reloj de la CPU y ofrece controles básicos de simulación.
- **Nivel de especificación**: Nivel medio. Representa una simplificación del procesador 8088 con arquitectura de 16 bits y memoria direccionable de 16 KiB.
- **Desarrollo del producto**: Nivel muy bueno. Licencia GNU Affero General Public License v3.0, última versión en 2020, con amplia evidencia de uso académico.
- **Cobertura de contenidos**: Nivel medio. Implementa dispositivos internos y externos, pero carece de visualización del ciclo de instrucción y métricas de rendimiento.

4.6.3 Emu8086

- **Usabilidad**: Nivel fácil. Inicialmente muestra el editor y permite activar los componentes del simulador a medida que se cargan programas.
- **Editor**: Nivel alto. Incluye notificaciones de errores de sintaxis, resaltado de código, puntos de interrupción y opciones para guardar/cargar programas.
- **Documentación**: Nivel completo. Ofrece un manual de instrucciones con ejemplos, un tutorial de aprendizaje y una guía de uso detallada.
- **Ejecución de simulación**: Nivel alto. Proporciona control avanzado de la simulación, como retroceder una instrucción ("step back").
- **Nivel de especificación**: Nivel completo. Detalla la arquitectura del procesador 8086, con memoria direccionable de 1 MiB y soporte para interrupciones de software y hardware.
- **Desarrollo del producto**: Nivel bueno. Licencia privativa, última actualización en 2023, desarrollado para plataformas de escritorio.
- **Cobertura de contenidos**: Nivel alto. Emula el booteo de una IBM PC desde un disco flexible (floppy disk) y soporta todos los modos de direccionamiento.

Tabla 4.1: Comparativa según criterios de evaluación preestablecidos

Criterio.de.Evaluacion	Simple.8.bit.Assembler.Simulator	VonSim	Emu8086
Usabilidad	Medio	Medio	Fácil
Editor	Bajo	Medio	Alto
Documentación	Mínima	Media	Completa
Ejecución de simulación	Medio	Medio	Alta
Nivel de especificación x86	Mínima	Media	Completa
Características del producto	Buena	Muy buena	Buena
Cobertura de contenidos	Baja	Media	Alta

4.7 Resultados

La asignatura promueve el uso de simuladores para apoyar la enseñanza y el aprendizaje, permitiendo aplicar los contenidos desarrollados en máquinas reales. Emu8086 es la herramienta más adecuada para esta finalidad, ya que facilita la implementación de programas en hardware real. Sin embargo, su dependencia de MS-DOS complica su ejecución en sistemas operativos actuales, requiriendo el uso de emuladores de MS-DOS, lo que añade complejidad al proceso de enseñanza y aprendizaje.

Desde 2018, la asignatura utiliza la versión 4.08 de Emu8086. La herramienta tiene un periodo de evaluación gratuito de 14 días, después del cual se debe adquirir una

licencia. Esto es un inconveniente, ya que se busca que los estudiantes puedan acceder a las herramientas de forma libre y gratuita.

Utilizar lenguaje NASM (Netwide Assembler) garantiza soporte tanto para Linux como Windows a través de herramientas libres como GCC (GNU Compiler Collection), generando programas para la arquitectura x86 de 16, 32 y 64 bits.

Emu8086 destaca por su interfaz dinámica, que muestra componentes como la pila, flags, teclado y pantalla solo cuando son necesarios, a diferencia de otros simuladores que presentan todos sus componentes desde el inicio.

En el criterio de evaluación dos, Emu8086 se destaca por su editor, que incluye puntos de ruptura para detener la ejecución del programa en un punto específico y retroceder a la intruccion anterior. Además, permite guardar y recuperar programas directamente desde el editor.

En el criterio de evaluación tres, Emu8086 se destaca por ofrecer tres tipos de documentación: un repertorio de instrucciones con ejemplos para cada tipo de instrucción, un manual que explica las partes de la herramienta y un tutorial para aprender a programar en ensamblador.

En el criterio de evaluación cuatro, Emu8086 se destaca por ofrecer una mayor cantidad de controladores para gestionar el flujo de ejecución, como la capacidad de retroceder la ejecución de una instrucción y recargar el programa actual.

En el criterio de evaluación cinco el emu8086 se destaca del resto debido a que ofrece una mayor especificidad de la arquitectura x86, además implementa interrupciones del sistema operativo MS-DOS, a través del cual se pueden ejecutar los programas en una máquina real.

En el criterio de evaluación seis VonSim se destaca del resto debido a que es licencia libre y posee una comunidad que respalda el proyecto.

En cuanto al último criterio, ninguna de las herramientas evaluadas cubre todos los contenidos que se pretende desarrollar con la ayuda de una herramienta, quedando excluido pasos del ciclo de instrucción y medidas de rendimientos (tiempo de CPU y CPI: ciclo por instrucción).

En resumen: - Emu8086 se destaca por su alta usabilidad, documentación completa, y precisión en la simulación de la arquitectura para la enseñanza de la arquitectura x86. Sin embargo, su licencia privativa y dependencia de MS-DOS representan desafíos importantes. - VonSim ofrece una buena solución con licencia libre y una interfaz más amigable, pero su nivel de especificación y cobertura de contenidos es medio, lo cual podría limitar su efectividad en el curso. - Simple 8-bit Assembler Simulator tiene limitaciones significativas en usabilidad, documentación, y especificación, lo que lo hace menos adecuado para una enseñanza completa de la arquitectura x86.

Del análisis realizado, se concluye que, aunque todos los simuladores evaluados presentan ventajas y desventajas, ninguno de los simuladores analizados parece adecuado para la asignatura sugiere que los simuladores actuales no cumplen completamente con los requisitos de complejidad y cobertura necesarios para la enseñanza exhaustiva de la arquitectura x86. Se recomienda usar Emu8086 mientras se desarrolla un simulador que combine sus mejores características con una licencia libre, con soporte para sistemas operativos modernos y una mayor cobertura de contenidos curriculares para la asignatura.

4.7.1 Publicación

Este análisis comparativo fue publicado en el XVII Congreso de Tecnología en Educación y Educación en Tecnología (2022), bajo el título "Herramientas de software para dar soporte en la enseñanza y aprendizaje de la arquitectura x86"[35].

Además, durante este proceso se estableció contacto con un desarrollador de VonSim, logrando implementar mejoras significativas, como animaciones de ejecución y documentación en línea, disponibles en su última versión publicada en agosto de 2023.

Capítulo 5

Diseño y Construcción del Simulador

En este capítulo se describe el diseño y desarrollo de una herramienta de simulación específica para la arquitectura x86, orientada a facilitar la enseñanza de los principios de arquitectura de computadoras. Se detalla la justificación del diseño, los pasos seguidos para su construcción y los métodos utilizados para validar su funcionalidad.

En el capítulo anterior se analizaron y evaluaron las herramientas de simulación existentes para la arquitectura x86. Esta revisión exhaustiva permitió identificar las limitaciones de las soluciones actuales y fundamentar la necesidad de desarrollar una herramienta específica, cuyos resultados fueron publicados posteriormente.

Para cumplir con este objetivo, el diseño de la herramienta sigue una serie de requisitos establecidos que guían su desarrollo, validación y alineación con los objetivos pedagógicos.

5.1 Requisitos de la herramienta

La herramienta de simulación está diseñada para cumplir los siguientes requisitos:

1. **Representación global de la estructura y funcionamiento de la computadora:**
   - Mostrar de manera visual la estructura completa de la computadora (CPU, bus, memoria y E/S) durante la ejecución de programas, resaltando relevantes.
2. **Soporte para la generación y ejecución de programas en ensamblador:**

- Facilitar la ejecución de programas en ensamblador paso a paso como completa, promoviendo la comprensión detallada de cada instrucción.

3. **Repertorio de instrucciones reducido y habilitación progresiva:**
   - Seleccionar un subconjunto esencial de instrucciones x86, que se habiliten progresivamente según avance la asignatura, adaptándose al ritmo de aprendizaje de los estudiantes.
4. **Simulación visual e interactiva de micropasos de instrucciones:**
   - Representar el flujo de datos con el lenguaje de transferencia entre registros (RTL), destacando visualmente cada componente involucrado en la ejecución de un programa.
5. **Gestión básica de interrupciones y periféricos:**
   - Incorporar un vector de interrupción predefinido para interactuar con dispositivos como teclado y monitor. También incluir instrucciones IN y OUT, junto con un módulo simplificado de entrada/salida.
6. **Métricas de rendimiento:**
   - Proporcionar indicadores clave como tiempo de ciclo, tiempo de CPU y ciclos por instrucción (CPI) para analizar la ejecución de programas.

5.2 Diseño conceptual

En esta sección se detalla el proceso de planificación del simulador, enfocándose en cómo los requisitos definidos orientan su construcción y garantizan la alineación con los objetivos pedagógicos establecidos.

5.2.1 Modelo arquitectónico

La arquitectura propuesta corresponde a una versión simplificada de **8 bits** de la arquitectura **x86**, diseñada con un repertorio de instrucciones reducido y un conjunto de componentes esenciales. Este diseño simplificado busca facilitar que los estudiantes comprendan conceptos clave como el formato de instrucciones, los modos de direccionamiento y el ciclo de búsqueda y ejecución.

Principales bloques funcionales

El simulador se compone de los siguientes bloques funcionales:

Tabla 5.1: Bloques funcionales principales

Bloque Funcional	Descripción
Unidad Central de Procesamiento (CPU)	Simulación de registros, Unidad de Control (UC) y Unidad Aritmética y Lógica (ALU).
Memoria	Estructura de memoria y simulación de operaciones de lectura y escritura.
Sistema de Entrada/Salida (E/S)	Interacción con periféricos y manejo de interrupciones.
Bus de datos, direcciones y control	Modelado de la comunicación entre componentes.

Características del CPU

El diseño del CPU simplificado se describe en la siguiente tabla 5.2:

Tabla 5.2: Componentes del CPU x86

Componente	Características
Arquitectura	Von Neumann, memoria compartida para datos e instrucciones.
Registros	4 registros de propósito general (‘AL‘, ‘BL‘, ‘DL‘ y ‘DL‘) de 8 bits cada uno.
	4 registros de propósito específico: • ‘IP‘ (Instruction Pointer) • ‘IR‘ (Instruction Register) • ‘SP‘ (Stack Pointer) • ‘RS‘ (registro de estado)
	2 registros intermedios (‘MAR‘: buffer de direcciones y ‘MBR‘: buffer de datos) para comunicación entre CPU y memoria.
Memoria	Memoria de 256 posiciones, con una capacidad de un byte por posición.
Buses	Bus de datos y direcciones de 8 bits.
Instrucciones	Longitud variable de instrucciones (1, 2 y 3 Bytes). Modos de direccionamiento: registro a registro, directo, indirecto e inmediato.

5.3 Implementación del repertorio de instrucciones

5.3.1 Selección del repertorio inicial

El repertorio se diseñó como una simplificación de la arquitectura x86, orientada a mejorar la enseñanza y el aprendizaje. En una primera etapa, se introducen únicamente las instrucciones esenciales para abordar los contenidos iniciales de la asignatura:

Tabla 5.3: Tabla de instrucciones y códigos de operación

Instrucciones	Código operación	nemónico	Acción
Transferencia de datos	0, 1, 2	MOV	• 0 - Operación entre registros • 1 - Operación carga a registro • 2 - Operación almacena en memoria
Pocesamiento de datos	3, 4, 5	ADD	3-4-5 Sumar
	6, 7, 8	SUB	6-7-8 Restar
	9, 10, 11	CMP	9-10-11 Comparar
Control de flujo	12	JMP	12-a Salto incondicional
		JZ	12-b Salto condicional si Z=1
		JC	12-c Salto condicional si C=1

5.3.2 Justificación de la simplificación

La arquitectura x86 destaca por su complejidad debido a su extenso repertorio de instrucciones y características avanzadas. Por ello, el simulador se desarrolla bajo un enfoque pedagógico que considera:

- **Reducir la carga cognitiva**: la simplificación del repertorio y de los componentes permite a los estudiantes enfocarse en principios fundamentales.
- **Aprendizaje progresivo**: se adopta un enfoque escalonado, empezando con un modelo simplificado y avanzando hacia representaciones más completas de x86.
- **Claridad pedagógica**: las prácticas son manejables en términos de tiempo y esfuerzo, promoviendo un aprendizaje práctico sin frustraciones.

5.3.3 Beneficios de la simplificación

El diseño del simulador contribuye a:

- **Comprensión fundamental**: los estudiantes pueden enfocarse en el ciclo de instrucciones, interacción de componentes y flujo básico de datos.
- **Análisis crítico**: comparar el modelo simplificado con x86 real fomenta un aprendizaje reflexivo y profundo.
- **Experimentación práctica**: proporciona un entorno accesible para explorar conceptos y corregir errores.

5.3.4 Formato de instrucciones

El formato definido para las instrucciones incluye los siguientes casos tabla 5.4:

Tabla 5.4: Tabla de codificación de instrucciones

Tipo	Operación	Codificación	Parámetros
Transferencia y procesamiento	A: entre registros	_____XXYY	XX = Registro destino YY = Registro fuente
	B: Cargar a registro	_____XX00 MMMMMMMM	XX = Registro destino M = Dirección de memoria
	C: Almacenar en memoria	_____00YY MMMMMMMM	YY = Registro fuente M = Dirección de memoria
Control	D: control de flujo	_____ffff MMMMMMMM	ffff = funcionalidad M = Dirección de memoria

Considerando:

- `____`: Código de operación de la instrucción, número de 4 bits.
- `XX` o `YY`: Índices de registros, número entre `0` y `3`, cada índice es de 2 bits.
- `M`: Dirección de memoria, número de 8 bits.
- `ffff`: representan el comportamiento de la instrucción, número de 4 bits.

5.3.5 Modos de direccionamiento

Los modos de direccionamiento definidos en esta etapa son:

- Registro a registro (`Rx`,`Ry`): operandos son registros del procesador.
- Directo (`[M]`): operando es el contenido de una dirección de memoria.

Banco de registros

Registros (R)	Binario	Decimal
AL	00	0
BL	01	1
CL	10	2
DL	11	3

En esta primera etapa se implementan las siguientes instrucciones con estos modos de direccionamiento y formato de instrucciones:

Tabla 5.6: Tabla de Instrucciones y Codificación

#	Instrucción	Acción	Codificación
0	MOV Rx, Ry	Rx ← Ry	0000 XXYY
1	MOV Rx, [M]	Rx ← Mem[Dirección]	0001 00YY MMMMMMMM
2	MOV [M], Ry	Mem[Dirección] ← Rx	0010 XX00 MMMMMMMM
3	ADD Rx, Ry	Rx ← Rx + Ry	0011 XXYY
4	ADD Rx, [M]	Rx ← Rx + Mem[Dirección]	0100 00YY MMMMMMMM
5	ADD [M], Ry	Mem[Dirección] ← Mem[Dirección] + Ry	0101 XX00 MMMMMMMM
6	SUB Rx, Ry	Rx ← Rx - Ry	0110 XXYY
7	SUB Rx, [M]	Rx ← Rx - Mem[Dirección]	0111 00YY MMMMMMMM
8	SUB [M], Ry	Mem[Dirección] ← Mem[Dirección] - Ry	1000 XX00 MMMMMMMM
9	CMP Rx, Ry	Modifica flags de Rx - Ry	1001 XXYY
A	CMP Rx, [M]	Modifica flags de Rx - Mem[Dirección]	1010 00YY MMMMMMMM
B	CMP [M], Ry	Modifica flags de Mem[Dirección] - Ry	1011 XX00 MMMMMMMM
C	JMP M	IP ← Dirección	1100 0000 MMMMMMMM
C	JC M	Si flag C=1 entonces IP ← Dirección	1100 0001 MMMMMMMM
C	JZ M	Si flag Z=1 entonces IP ← Dirección	1100 0011 MMMMMMMM

5.3.6 Ciclo de la instrucción: Etapas de captación y ejecución

En este apartado se detalla el proceso llevado a cabo por la Unidad de Control (UC) para ejecutar cada instrucción, destacando los aspectos clave:

- Secuencia de pasos.
- Registros involucrados en cada etapa del proceso.
- Uso de los buses de datos, direcciones y control.
- Señales de control generadas.

El **ciclo de la instrucción** se define como la secuencia de microoperaciones necesarias para ejecutar una instrucción en el sistema. Estas microoperaciones se expresan mediante un lenguaje de transferencia entre registros, representado de la forma:

`destino` ← `origen`

Etapa de Captación:

En esta etapa, común a todas las instrucciones, consiste en la lectura de la instrucción desde la memoria.

1. `MAR` ← `IP`: el contenido del registro puntero de instrucciones `IP` se transfiere al registro de direcciones de memoria `MAR`. La UC genera la señal necesaria para seleccionar el valor del `IP` y copiarlo en el `MAR`.
2. `MDR` ← `read(Memoria[MAR]) ; IP` ← `IP` + **1**: la UC activa la señal de lectura (read) hacia la memoria, utilizando el valor del `MAR` como dirección. El dato leído se transfiere al registro de datos de memoria `MBR` a través del bus de datos. Simultáneamente, el `IP` se incrementa en 1 para apuntar al siguiente byte.
3. `IR` ← `MBR`: el contenido del `MBR` se transfiere al registro de instrucciones `IR`, completando la etapa de captación.

Etapa de Ejecución:

En esta etapa, las operaciones específicas dependen del tipo de instrucción. A continuación, se describen algunos casos representativos:

- MOV `Rx`, `Ry` (Copia entre registros)

1. **`Rx`** ← **`Ry`**: el contenido del registro `Ry` se copia en el registro `Rx`.

- MOV `Rx`, `[Dirección]` (Carga a registro)

  1. **`MAR`** ← **`IP`**: el valor del `IP` se transfiere a `MAR`.
  2. **`MBR`** ← **`read(Memoria[MAR]); IP`** ← **`IP + 1`**: se lee (read) de memoria el contenido de la dirección indicada por `MAR` y se almacena en `MBR`.Simultáneamente, el `IP` se incrementa.
  3. **`MAR`** ← **`MBR`**: el contenido de `MBR` se transfiere a `MAR`.
  4. **`MBR`** ← **`read(Memoria[MAR])`**: se lee de memoria el contenido de la dirección indicada por `MAR` y se almacena en `MBR`.
  5. **`Rx`** ← **`MBR`**: el contenido del `MBR` se copia al registro `Rx`.

- MOV `[Dirección]`, `Ry` (Almacenar en memoria)

  1. 2. 3. igual que MOV `Rx`, `[Dirección]`.
  2. **`MBR`** ← **`Ry`**: el contenido de `Ry` se transfiere a `MBR`.
  3. **`write(Memoria[MAR])`** ← **`MBR`**: el contenido de `MBR` se escribe (write) en memoria en la dirección apuntada por el `MAR`.

- ADD `Rx`, `[Dirección]` (Sumar a registro)

  1. 2. 3. 4. igual que MOV `Rx`, `[Dirección]`.
  2. **`Rx`** ← **`Rx + MBR`**: la Unidad Aritmético-Lógica (ALU) realiza la suma entre `Rx` y `MBR`, almacenando el resultado en `Rx`. El Registro de Estado `RS` se actualiza con los indicadores correspondientes.

- SUB `[Dirección]`, `Ry` (Restar a memoria)

  1. 2. 3. 4. igual que MOV `Rx`, `[Dirección]`.
  2. **`MBR`** ← **`Ry - MBR`**: la ALU resta el contenido de `MBR` al de `Ry`, almacenando el resultado en `MBR`. El `RS` se actualiza.
  3. **`write(Memoria[MAR])`** ← **`MBR`**: el contenido de `MBR` se escribe en memoria en la dirección apuntada por el `MAR`.

- CMP `Rx`, `[Dirección]` (Comparar a registro)

  1. 2. 3. 4. igual que MOV `Rx`, `[Dirección]`.
  2. **`Rx - MBR`**: la ALU realiza la resta entre el contenido de `Rx` y `MBR`. Aunque el resultado no se almacena, el `RS` se actualiza con los indicadores de comparación.

- JMP `Dirección` (Salto incondicional)

  1. 2. igual que MOV `Rx`, `[Dirección]`.
  2. **`IP`** ← **`MBR`**: El contenido del `MBR` se transfiere al registro `IP`, estableciendo la nueva dirección de ejecución.

- JC `Dirección` (Salto condicional si C=1)

1. 2. Igual que MOV `Rx`, `[Dirección]`.
2. `Si C` == 1: `IP` ← `MBR`: si la bandera de acarreo (`C`) está activa, el contenido del `MBR` se transfiere al registro `IP`.

- JZ `Dirección` (Salto condicional si Z=1)
  1. 2. igual que MOV `Rx`, `[Dirección]`.
  2. `Si Z` == 1: `IP` ← `MBR`: si la bandera de cero (`Z`) está activa, el contenido del `MBR` se transfiere al registro `IP`.

5.3.7 Modos de direccionamiento adicionales

Se incoporan los siguientes modos de direccionamiento:

- Indirecto por registro (`[BL]`): la dirección del operando se encuentra en el registro `BL` del procesador.
- Inmediato (`d`): operando es un valor contenido en la instrucción.

5.3.8 Formato ampliado

Las instrucciones ahora se codifican con 1, 2 o 3 bytes dependiendo del modo:

Las instrucciones de cargar a registro (B):

Función	Byte	Interpretación
00	2	Directo `M` = Dirección de memoria
01	1	Indirecto utiliza como operando implícito el registro `BL` y no requiere operando `M`
10	2	Inmediato `d` = Dato inmediato
11	2	Indirecto la dirección se calcula operando implícito `BL` + Dato inmediato

El formato ampliado para las instrucciones incluye los siguientes casos tabla 5.8:

Modo direccionamientos C: Almacenar en memoria		
`ffff`= funcionalidad	**`Byte`= tamaño**	**Interpretación**

Modo direccionamientos C: Almacenar en memoria		
00YY	2	directo D = Dirección de memoria, YY = Registro fuente
01YY	1	indirecto BL, YY = Registro fuente
01YY	2	Indirecto la dirección se calcula operando implícito BL + Dato Inmediato
1100	3	Inmediato a memoria
1101	2	Inmediato a memoria mediante indirecto BL
1110	3	Inmediato a memoria mediante indirecto BL+ Dato Inmediato

#	Instrucción	Acción	Codificación
0	MOV Rx, Ry	Rx ← Ry	0000 XXYY
1	MOV Rx, [M]	Rx ← Mem[Dirección]	0001 XX00 MMMMMMMM
1	MOV Rx, [BL]	Rx ← Mem[BL]	0001 XX01
1	MOV Rx, D	Rx ← Dato	0001 XX10 MMMMMMMM
1	MOV Rx, [BL + D]	Rx ← Mem[BL + Dato]	0001 XX11 MMMMMMMM
2	MOV [M], Ry	Mem[Dirección] ← Rx	0010 00YY MMMMMMMM
2	MOV [BL], Ry	Mem[BL] ← Rx	0010 01YY
2	MOV [BL + D], Ry	Mem[BL + Dato] ← Rx	0010 10YY MMMMMMMM
2	MOV [M], D	Mem[Dirección] ← Dato	0010 1100 MMMMMMMM ddddd
2	MOV [BL], D	Mem[BL] ← Dato	0010 1101 MMMMMMMM
2	MOV [BL + D], D	Mem[BL + Dato] ← Dato	0010 1110 MMMMMMMM
3	ADD Rx, Ry	Rx ← Rx + Ry	0011 XXYY
4	ADD --, --	Mismo direccionamientos que MOV	0100 ---- --------
5	ADD --, --	Mismo direccionamientos que MOV	0101 ---- -------- -----
6	SUB Rx, Ry	Rx ← Rx - Ry	0110 XXYY
7	SUB --, --	Mismo direccionamientos que MOV	0111 ---- --------
8	SUB --, --	Mismo direccionamientos que MOV	1000 ---- -------- -----
9	CMP Rx, Ry	Modifica *flags* de Rx - Ry	1001 XXYY
A	CMP --, --	Mismo direccionamientos que MOV	1010 ---- --------
B	CMP --, --	Mismo direccionamientos que MOV	1011 ---- -------- -----
C	JMP M	IP ← Dirección	1100 0000 MMMMMMMM
C	JC M	Si flag C=1 entonces IP ← Dirección	1100 0001 MMMMMMMM
C	JZ M	Si flag Z=1 entonces IP ← Dirección	1100 0011 MMMMMMMM

#	Instrucción	Acción	Codificación
C	`Jxx M`	Se pueden implementar más flags y CALL	`1100 ffff --------`

5.3.9 Ciclo de la instrucción: modos adicionales

- MOV `Rx`, `[BL]` (Carga indirecto)
  1. **`MAR`** ← **`IP`**: El valor del `IP` se transfiere a `MAR`.
  2. **`MBR`** ← **`read(Memoria[MAR]); IP`** ← **`IP + 1`**: Se lee (read) de memoria el contenido de la dirección indicada por `MAR` y se almacena en `MBR`.Simultáneamente, el `IP` se incrementa.
  3. **`MAR`** ← **`BL`**: El contenido de `BL` se transfiere a `MAR`.
  4. **`MBR`** ← **`read(Memoria[MAR])`**: Se lee de memoria el contenido de la dirección indicada por `MAR` y se almacena en `MBR`.
  5. **`Rx`** ← **`MBR`**: El contenido del `MBR` se copia al registro `Rx`.
- MOV `Rx`, `Dato` (Carga valor inmediato)
  1. **`MAR`** ← **`IP`**: El valor del `IP` se transfiere a `MAR`.
  2. **`MBR`** ← **`read(Memoria[MAR]); IP`** ← **`IP + 1`**: Se lee (read) de memoria el contenido de la dirección indicada por `MAR` y se almacena en `MBR`.Simultáneamente, el `IP` se incrementa.
  3. **`Rx`** ← **`MBR`**: El contenido del `MBR` se copia al registro `Rx`.

5.4 Tercera etapa

En esta etapa se incorporaron las instrucciones de manejo de pila y de interrupciones, ampliando la funcionalidad del simulador para cubrir un conjunto más completo de operaciones propias de la arquitectura x86. A continuación, se detalla el conjunto completo de instrucciones, incluyendo su clasificación, nemónicos y las acciones que representan.

Tabla 5.11: Tabla de Instrucciones y Códigos de Operación de la Arquitectura x86

Código operación	**Instrucciones**	**nemónico**	**Acción**
MOV	Transferencia de datos	MOV destino, origen	1- Copiar entre registros 2- Cargar a registro 3- Almacenar en memoria

ADD	Aritmética	ADD destino, origen	1- Sumar 2- Restar 3- Comparar
JMP	Control de flujo	JMP destino	Salto incondicional JMP. Saltos condicionales Jxx.Llamadas a rutinas CALL y retorno RET
PUSH, POP, OUT, IN	Pila y E/S	PUSH, POP, OUT, IN	Poner en la pila PUSH Retirar de la pila POP Enviar un byte al puerto del dispositivo de E/S Recibir un byte del dispositivo de E/S
INT , IRET	Interrupciones	INT , IRET	Llamar a una rutina de tratamiento de interrupción INT Retornar de una rutina de tratamiento de interrupción IRET
NOP , HLT	Control	NOP , HLT	No opera NOP Detiene el CPU HLT

5.5 Simulación visual e interactiva

Esta sección detalla la implementación de las características visuales e interactivas del simulador, diseñadas para facilitar la comprensión del flujo de datos y las microoperaciones dentro de la arquitectura x86.

5.5.1 Representación gráfica de componentes

- **Diseño de la interfaz visual**: incluye elementos como registros, buses y memoria, presentados de manera clara y organizada.
- **Resaltado dinámico**: los componentes relevantes se destacan visualmente durante la ejecución, proporcionando un seguimiento en tiempo real del flujo de datos.

Tabla 5.8: Tabla de codificación de instrucciones ampliado

Tipo	Operación	Codificación	Parámetros
Transferencia y procesamiento	A: entre registros	_____XXYY	XX = Registro destino, YY = Registro fuente
	B: Cargar a registro	_____XX00 MMMMMMMM	XX00 = Registro destino y M = Dirección de memoria
		_____XX01	XX01 = Registro destino y direccion en registro [BL]
		_____XX00 dddddddd	XX10 = Registro destino y d = Dato inmediato
	C: Almacenar en memoria	_____00YY MMMMMMMM dddddddd	YY = Registro fuente, 'M' = Dirección de memoria, 'd' = Dato Inmediato
Control	D: control de flujo	_____ffff MMMMMMMM	ffff = funcionalidad, M = Dirección de memoria

Nota:
d = dato inmediato, no puede ser destino de la instrucción.

5.5.2 Ejecución paso a paso

- **Flujo interactivo**: permite avanzar por cada etapa del ciclo de instrucción, desde la captura hasta la finalización.
- **Opciones de visualización**: se pueden observar los micropasos que conforman cada etapa de la ejecución, fomentando una comprensión detallada del proceso.

5.6 Gestión de interrupciones y periféricos

Se describe la simulación de interrupciones y operaciones de entrada/salida, proporcionando una implementación básica para la interacción con periféricos.

5.6.1 Manejo del vector de interrupciones

- **Vector predefinido**: se incluye un conjunto de interrupciones estándar.
- **Simulación de interrupciones**: ejemplos como las interacciones con teclado y pantalla están modeladas para su análisis práctico.

5.6.2 Instrucciones IN y OUT

- **Operaciones de entrada/salida**: se implementan y simulan las instrucciones básicas para la interacción con periféricos.

5.6.3 Segunda etapa

En esta etapa, se amplían los modos de direccionamiento e instrucciones:

5.7 Integración de métricas de rendimiento

El simulador incorpora métricas clave para analizar el impacto del rendimiento en la arquitectura de computadoras.

5.7.1 Métricas calculadas

- **Indicadores**: tiempo de ciclo, tiempo de CPU y ciclos por instrucción (CPI).
- **Visualización**: las métricas se presentan en tiempo real durante la simulación, facilitando su análisis.

5.7.2 Análisis de casos de estudio

- Se incluyen ejemplos que ilustran cómo interpretar las métricas para optimizar el rendimiento.

5.8 Proceso de validación

El proceso de validación garantiza que el simulador cumple con los requisitos pedagógicos y funcionales.

5.8.1 Validación funcional

- **Pruebas de módulos**: cada componente se verificó de manera individual.
- **Ejecución completa**: programas de prueba comprobaron la correcta implementación de las instrucciones.

5.8.2 Evaluación pedagógica

- **Retroalimentación**: opiniones de estudiantes y docentes guiaron los ajustes realizados.

- **Objetivos educativos**: el simulador prioriza la claridad conceptual sin sacrificar la precisión técnica.

5.9 Portabilidad y Mantenibilidad

- **Portabilidad**: la herramienta se implementa como una aplicación web, compatible con cualquier navegador moderno.
- **Mantenibilidad**: el código modular y bien documentado facilita futuras actualizaciones y mejoras.

Bibliografía

[1] M. A. Colombani, M. A. Falappa, A. G. Delduca, and J. M. Ruiz, "PID novel 7065: Enseñanza/aprendizaje de asignatura Arquitectura de Computadoras con herramientas de simulación de sistemas de cómputos." Feb. 2022. Accessed: Jul. 10, 2024. Available: https://proyectos.uner.edu.ar/aplicacion.php?ah=st668e6d47663eb&ai=gestion_extinv%7C%7C23000105

[2] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-event system simulation*, 5th ed. Prentice Hall, 2010.

[3] A. M. Law, *Simulation Modeling & Analysis*, 5th ed. New York, NY, USA: McGraw-Hill, 2015.

[4] S. Robinson, *Simulation: The Practice of Model Development and Use*, 2nd edition. 2014.

[5] C. Lion, "Los simuladores. Su potencial para la enseñanza universitaria," *Cuadernos de Investigación Educativa*, vol. 2, no. 12, pp. 53–66, 2005.

[6] G. Contreras, R. G. Torres, and M. S. R. Montoya, "Uso de simuladores como recurso digital para la transferencia de conocimiento," *Apertura: Revista de Innovación Educativa*, vol. 2, no. 1, pp. 86–100, 2010.

[7] A. Garcia-Garcia, J. C. Saez, J. L. Risco-Martin, and M. Prieto-Matias, "PBBCache: An open-source parallel simulator for rapid prototyping and evaluation of cache-partitioning and cache-clustering policies," *Journal of Computational Science*, vol. 42, p. 101102, 2020.

[8] B. Nova, J. C. Ferreira, and A. Araújo, "Tool to support computer architecture teaching and learning," in *Engineering Education (CISPEE), 2013 1st International Conference of the Portuguese Society for*, IEEE, 2013, pp. 1–8.

[9] B. Mustafa, "Evaluating A System Simulator For Computer Architecture Teaching And Learning Support," *Innovation in Teaching and Learning in Information and Computer Sciences*, vol. 9, no. 1, pp. 100–104, 2010, doi: 10.11120/ital.2010.09010100.

[10] Z. Radivojevic, M. Cvetanovic, and J. Đordevic, "Design of the simulator for teaching computer architecture and organization," in *2011 Second Eastern European Regional Conference on the Engineering of Computer Based Systems*, IEEE, 2011, pp. 124–130.

[11] B. Nikolic, Z. Radivojevic, J. Djordjevic, and V. Milutinovic, "A Survey and Evaluation of Simulators Suitable for Teaching Courses in Computer Architecture and Organization," *IEEE Transactions on Education*, vol. 52, no. 4, pp. 449–458, Nov. 2009, doi: 10.1109/TE.2008.930097.

[12] R. Hasan and S. Mahmood, "Survey and evaluation of simulators suitable for teaching for computer architecture and organization Supporting undergraduate students at Sir Syed University of Engineering & Technology," in *Control (CONTROL), 2012 UKACC International Conference on*, IEEE, 2012, pp. 1043–1045.

[13] J. L. Hennessy and D. A. Patterson, *Computer architecture: A quantitative approach*, Fifth Edition. Elsevier, 2012.

[14] W. Stallings, *Computer organization and architecture: Designing for performance*, Eleventh Edition. Pearson, 2013.

[15] Intel, "64 and IA-32 architectures software developers manual," *325462-060US*, vols. 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C and 3D, p. 4670, 2016, Available: https://software.intel.com/en-us/articles/intel-sdm

[16] AMD, "Developer Guides, Manuals & ISA Documents." Apr. 2019. Accessed: Apr. 21, 2019. Available: https://developer.amd.com/resources/developer-guides-manuals/

[17] P. Abel, *IBM PC Assembly Language and Programming*, Fifth Edition. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2000.

[18] D. Skrien, "CPU Sim 3.1: A tool for simulating computer architectures for computer organization classes," *Journal on Educational Resources in Computing (JERIC)*, 2001.

[19] J. L. Peterson, *Petri net theory and the modeling of systems*. Prentice Hall PTR, 1981.

[20] B. Zeigler, H. Prähofer, and T. G. Kim, "Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems," vol. 2, Jan. 2000.

[21] B. P. Zeigler, A. Muzy, and E. Kofman, *Theory of Modeling and Simulation: Discrete Event & Iterative System Computational Foundations*. Academic Press, 2018.

[22] L. Null and J. Lobur, *Essentials of computer organization and architecture*. Jones & Bartlett Publishers, 2014.

[23] Patterson and others, *Computer organization and design: The hardware/software interface-5th*. Morgan Kaufmann, 2014.

[24] A. S. Tanenbaum, *Structured computer organization*. Pearson Education, 2013.

[25] M. J. Murdocca and V. Heuring, *Principles of computer architecture*. Pearson Education, 2000.

[26] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The risc-v instruction set manual, volume i: User-level isa," *CS Division, EECE Department, University of California, Berkeley*, p. 28, 2014.

[27] K. R. Irvine and L. B. Das, *Assembly language for x86 processors*. Prentice Hall, 2011.

[28] B. International, *Turbo assembler user's guide*. Borland International, 1993.

[29] M. Corporation, *Microsoft macro assembler 6.1 reference*. Microsoft Press, 1992.

[30] The NASM Project, *The netwide assembler (nasm) manual*. 2023.

[31] R. Hyde, *The art of assembly language*. No Starch Press, 2010.

[32] A. Stork, C.-A. Thole, S. Klimenko, I. Nikitin, L. Nikitina, and Y. Astakhov, "Towards interactive simulation in automotive design," *The Visual Computer*, vol. 24, pp. 947–953, 2008.

[33] F. Jentsch and M. Curtis, *Simulation in aviation training*. Routledge, 2017.

[34] F. A. Calvo Valdés, J. F. Roldán Ramírez, and A. San Miguel Sánchez, "Simulador del procesador MIPS sobre el formalismo DEVS," 2010, Accessed: Sep. 19, 2024. Available: https://hdl.handle.net/20.500.14352/46063

[35] M. A. Colombani, J. M. Ruiz, A. G. Delduca, and M. A. Falappa, "Herramientas de software para dar soporte en la enseñanza y aprendizaje de la arquitectura x86," 2022. Accessed: Jul. 10, 2024. Available: http://sedici.unlp.edu.ar/handle/10915/139908

[36] P. BEHROOZ, *Computer Architecture: From Microprocessors to Supercomputers*. Oxford University Press Inc, 2005.